ASC 2026 Proposal of XTU

Team members: Huayu Shen, Yangkun Pei, Long Jiang, Muyuan Chen, Tao Li
Advisor: Peiying Lin

Contents

1. Supercomputing Activities 4
1.1. The HPC Platform of XTU 4
1.2. HPC Activities 4
1.3. Campus Activities 5
1.4. Research and Applications 5
1.5. Key Achievements 6
2. Our Big Family 7
2.1. Setup of the team 7
2.2. Introduction of members 8
2.3. Team slogan 9
3. Design of our HPC System 10
3.1. Architectural Diagram Overview 10
3.2. System Configuration and Architecture Analysis 10
3.2.1. Hardware Configurations 10
3.2.2. Software Configurations 11
3.2.3. Interconnection Architecture 12
3.3. Computing Performance Description 13
3.4. Power Consumption Description 13
4. HPL Test and Optimization 15
4.1. Analysis of HPL 15
4.2. Performance Optimization Methods 15
4.2.1. Modification of "HPL.dat" 15
4.2.2. Third-Party Library 16
4.2.3. Accelerate Procedures by Using Appropriate Compiler Options 17
4.2.4. Multi-thread Acceleration 17
4.3. Machine Performance Analysis 18
4.4. Experimental Results 18
4.4.1. Software Setup 18
4.4.2. CPU Evaluation Results 19
4.4.3. GPU Evaluation Results 19
4.5. Summary 20

5. HPCG Test and Optimization 21
5.1. Analysis of HPCG 21
5.2. Performance Optimization Methods 21
5.2.1. Modification of "HPCG.dat" 21
5.2.2. Third-Party Library for HPCG 22
5.2.3. Parallelization Paradigms for HPCG 23
5.2.4. Hybrid Version of HPCG 24
5.3. Experimental Results 24
5.3.1. CPU Evaluation Results 24
5.3.2. GPU Evaluation Results 24
5.4. Summary 25
6. Optimization for UnifoLM-WMA-0 Inference 26
6.1. Introduction 26
6.2. Optimization Methods 26
6.2.1. NUMA Affinity Optimization 28
6.2.2. Mixed Precision 29
6.2.3. I/O Improvements 31
6.2.4. DeepCache Implementation 33
6.2.5. FlashAttention Recomputation 36
6.3. Experiments 38
6.3.1. Environment Setup 38
6.3.2. Evaluation Results 39
6.4. Optimization Plans 42
6.4.1. Operator Fusion 42
6.4.2. Multi-GPUs 43
6.5. Conclusion 44
7. Optimization for Numerical Relativity Code AMSS-NCKU 45
7.1. Introduction 45
7.2. Optimization Methods 45
7.2.1. Compiler Parameter Adjustment 46
7.2.2. Inline Function 47
7.2.3. Grid Partition Optimization 47
7.2.4. Memory Alignment 48
7.2.5. Non-blocking & Request Separation 49
7.2.6. Improvement of Cache Hit Ratio 49
7.2.7. Adjustment of MPI numbers 50

7.3. Experiments 50
7.3.1. Environment Setup 50
7.3.2. Installation Instructions. 51
7.3.3. Evaluation Result 52
7.4. Analysis and Optimization Plans 55
7.4.1. Hybrid MPI+OpenMP Parallelization Plan 56
7.5. Conclusion 56
Acknowledge. 58
References 59

1. Supercomputing Activities

Xiangtan University(XTU) is a prestigious comprehensive national key university and a "Double First-Class" construction university of the country. Standing majestically in the hometown of the great leader Mao Zedong, XTU is a place of great renown. XTU not only has modern teaching facilities, but also is equipped with an advanced high-performance computing center. In terms of high-performance talent cultivation, the school provides students with abundant high-performance learning resources, including cutting-edge professional courses, high-level academic lectures, innovative research projects, and a powerful high-performance computing platforms. These resources not only broaden students' horizons, but also provide them solid academic and technical support. Last year, we participated in the ASC competition for the first time. Although we didn't make it to the finals, our ranking of 46th was very encouraging for the students, and the school also gave us high recognition. Next, we will introduce the university's supercomputing platform and supercomputing-related activities.

1.1. **The HPC Platform of XTU.**

Xiangtan University's Super Heterogeneous Parallel Computing Cluster is a key project supported by the provincial finance under the "4th Five-Year Plan" for new infrastructure construction. The cluster includes ClusterEngine high-performance computing platform and AIstation artificial intelligence platform, and is open to all teachers and students of the university. The computing capacity of the cluster is the theoretical computing capacity of 509 trillion times of CPU double-precision peak performance (6,112 physical cores) and 1552 trillion times of GPU single-precision theoretical computing capacity. The overall logical framework design diagram of the cluster is shown in the figure.

The ClusterEngine high-performance computing platform consists of 1 management node, 1 login node, 1 physical monitoring platform, 85 general-purpose computing nodes, 2 large-memory computing nodes, totaling 5472 CPU cores and a total memory capacity of 44.5TB. The nodes are interconnected through Gigabit Ethernet and Infiniband. The management and login nodes are each equipped with 16 CPU cores and 64GB of memory.

The cluster has two resource partitions: CU and FAT partitions. The CU partition is configured with 80 general-purpose computing nodes, each with 64 Intel 8358 CPU cores and 512GB of memory, totaling 5,120 CPU cores. The FAT partition is configured with 2 large-memory computing nodes, each with 16 Intel 6334 CPU cores and 1TB of memory.

The main function of the platform is to provide high-performance scientific and engineering computing services to the entire university, meeting the needs of various disciplines for large-scale data processing and large-scale scientific computing. The platform's main application areas include providing high-performance computing environments for mathematics, statistics, mechanics, physics, chemistry, chemical engineering, biology, earth science, aerospace science and technology, computer science, and many other disciplines. The platform supports a wide range of application software, such as MATLAB, OpenFOAM, ABINIT,GROMACS, NAMD and so on.

The AIstation artificial intelligence platform consists of 10 GPU computing nodes, with a total of 640 CPU cores, 80 NVIDIA A100 acceleration cards, a total memory capacity of 5TB, and shared with the parallel storage and high-performance computing platform. The total capacity is 1.3PB. The GPU's measured peak speed can reach 888.9 trillion operations per second. This platform supports single-machine and distributed running of mainstream AI frameworks as well as task visualization.

1.2. **HPC Activities.**

Xiangtan University actively conducts academic activities and research projects related to supercomputing. For instance, in June 2022, Professor Peng Shaoliang, the deputy director of the National Supercomputing Changsha Center, delivered an academic report titled "Prospects of Computational Power and Applications of the Metaverse Based on Supercomputing" at Xiangtan University. Moreover, the university has participated in the construction of the Hunan Computing Power Network Integration Collaborative Innovation Platform, promoting the interconnection and networking of computing infrastructure and the efficient utilization of computing resources. Furthermore, following our success in last year's supercomputing competition, the university's official website[1], the student affairs department's official WeChat account[2], and the Hunan Daily[3] all published relevant reports, demonstrating the university's strong support for the ASC supercomputing competition. Through these supercomputing platforms and related activities, Xiangtan University not only provides students with abundant learning resources but also helps the university to carry out scientific research innovation and talent cultivation in the field of high-performance computing.

1.3. **Campus Activities.**

To cultivate talents in the field of high-performance computing, the school provides various activities. In terms of courses, the relevant courses offered by the school include "Parallel Programming", "Computer Organization Principles", "Syntax and Semantics of Compilers", "Computer System Architecture", "Operating Systems", "Principles of Computer Networks", "Linux System Analysis and Programming", "Algorithm Design and Analysis", "Mathematical Analysis", "C Programming", "Graph Theory and Combinatorial Mathematics", "Data Structures", "Artificial Intelligence", etc., which are a wealth of courses. In terms of regular training, through platforms such as EasyHPC, the school invites previous ASC participants to provide regular practice and training for students. Separate classrooms and servers are also provided for students. In terms of interest groups, in addition to forming ASC competition teams, the school's ACM team also actively participates.

1.4. **Research and Applications.**

The research and application achievements of XTU in the field of supercomputing are remarkable, including parallel solvers, the development of high performance computing software, the optimization of computer system architecture, the design of high performance computing algorithms and the application of high performance computing in scientific research, engineering design, industrial production and other fields. The following is only a brief list.

For example, the JXPAMG solver jointly developed with the Beijing Institute of Applied Physics and Computational Mathematics has become the default linear solver of JASMIN, a high-performance parallel computing software platform, with its algorithm innovation, efficient parallel performance and multi-field applicability, and its comprehensive performance is better than the international popular similar solvers. The solver has greatly improved the computational efficiency of some defense application model problems, and has an important influence in the field of scientific calculation and engineering simulation.

The chemical flooding model is widely used in the numerical simulation of complex reservoir because of its high oil recovery. Therefore, for the numerical simulation of chemical flooding reservoir, CPU-GPU heterogeneous parallel discretization technology is used to discretized the chemical flooding model. The numerical experiment results show that the CPU-GPU heterogeneous parallel

[1]https://www.xtu.edu.cn/info/1123/22352.htm

[2]https://m.voc.com.cn/xhn/news/202503/28109152.html

[3]https://mp.weixin.qq.com/s/dcxK75riuyZ6Pn77fU4OBw

discretization method has a good acceleration effect. For example, for the calculation example with a grid size of about 1.25 million, The speed of the new algorithm is 3.99 times faster than that of the serial algorithm. Then, a non-smooth aggregation algebraic multi-grid preconditioned conjugate gradient iterative algorithm (UA-AMG-CG) based on CPU-GPU heterogeneous system with low operator complexity, high acceleration ratio and good robustness is designed for water phase pressure equation and component concentration equation, and the corresponding program module is developed.

1.5. **Key Achievements.**

As mentioned above, XTU's research in supercomputing includes many aspects, and have published multiple papers in international journals and conferences in the fields of mathematics and computer science and so on. Such as *CCF Transactions on High Performance Computing, Applicable Analysis, Mathematical Methods in the Applied Sciences, International Conference on Management of Data(SIGMOD) International Conference on Very Large Data Bases(VLDB), IEEE Transactions on Parallel and Distributed Systems(TPDS)* and other journal conferences, here is a brief overview of two of the outcomes:

Innovation 1: Parallel Reservoir Simulators on Tianhe-2 [1].

Parallel reservoir simulators are now widely used with availability of super computers. Modern massively parallel supercomputers demonstrate great power for simulating large-scale reservoir models. However, improving scalability and efficiency for fully implicit methods on emerging parallel architectures is still challenging. In this work, we present a robust discretization together with a parallel linear solver algorithm; and we explore the parallel implementation on the world's fastest supercomputer Tianhe-2. Starting with a general compositional model, we focus on the black oil model and developed Parallel eXtension Framework for parallelizing the serial simulator. A parallel preconditioner based on fast auxiliary space preconditioning (FASP) is applied to solve the Jacobian system arising from the fully implicit discretization. The parallel simulator was validated using large-scale black oil benchmark problems, for which parallel scalabilities were tested. Giant reservoir models with over 100 million grid blocks have been simulated within a few minutes, and test the strong scalability of AMG solver with 1 billion unknown. We also demonstrate the parallelization and acceleration using Intel Xeon Phi coprocessors. In the end, the efficiency of the parallel simulator is illustrated by a giant reservoir using up to 10,000 cores, for which the CPU and communication time are summarized for the linear and nonlinear algorithms.

Innovation 2: Efficient Algorithms for the Maxwell Eigenvalue Problem [2]. The Maxwell eigenvalue problem is a basic and important computational model in computational electromagnetism, e.g., in regard to electromagnetic waveguides and resonances in cavities. This work proposed two two-grid methods for the Maxwell eigenvalue problem. The new two-grid schemes reduce the solution of the Maxwell eigenvalue problem on a fine grid to one linear indefinite Maxwell equation on the same fine grid and an original eigenvalue problem on a much coarser grid. The new schemes, therefore, save total computational cost. The error estimates reveals that the two-grid methods maintain asymptotically optimal accuracy, and the numerical experiments presented confirm the theoretical results.

2. Our Big Family

Under the guidance of our advisors, and after relevant training and selection processes, the ASC competition team was formed. As the second ASC competition team of school, we underwent a lengthy preparation period and received assistance from many people both inside and outside the school, including members of the previous ASC team. We are grateful for their technical support and valuable suggestions, which enabled us to participate in this competition successfully. The following sections detail the team composition and current status.

2.1. **Setup of the team.**



Figure 1. *Training Time*

This competition was initiated by tutor, Peiying Lin, who teaches *Parallel Programming* and *Computer Organization Principles*. Peiying achived Ph.D. from Hunan University and is engaged in high-performance computing research. Starting in September 2024, Lin promoted the ASC competition through courses and lectures at the university and developed the selection criteria. For this year's competition, starting in September 2025, new selection criteria were established, and our current team was formed. Since the team's formation, Lin has organized specialized training sessions on fundamental knowledge, parallel computing, and optimization techniques, and has repeatedly

invited former ASC competitors from both within and outside the university to provide guidance and share their experiences as shown in Figure 1.

2.2. **Introduction of members.**

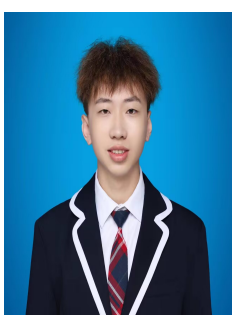
Our participating team consists of five official members and several reserve members. The instructor is Peiying Lin. The participating students include undergraduate students majoring in Computer Science and Technology and Cyber Security. They are Huayu Shen, Yangkun Pei, Long Jiang, Tao Li (majoring in Computer Science and Technology, junior year) and Muyuan Chen (majoring in Cyber Security, sophomore year). And, the photo of our team is shown in Figure **??**. The following is an introduction of each team member.



Figure 2. *Team Members*



Huayu Shen, the team leader. He possesses outstanding abilities to handle difficulties, solid knowledge in computer science, and a courageous spirit of exploration. He is proficient in the Linux system and Git version control tools, and also skilled in MPI and OpenMP multi-threading programming. He once participated in the development of an online learning platform as well as an intelligent course scheduling system based on genetic algorithms and multimodal optimization. In this ASC competition, he not only managed the overall progress but also participated in the optimization of HPL tests and the performance optimization of AMSS-NCKU.



Yangkun Pei is good at C/C++, Python and Shell scripting for cross-platform development, familiar with MPI, OpenMP and CUDA GPU acceleration technologies. He possesses the ability to design and optimize multi-level parallelization algorithms. He is also familiar with the Linux development environment, CMake

build toolchain and advanced optimization options of GCC/Intel compilers. In this competition, he is responsible for the deployment and operation of AMSS-NCKU and the design and implementation of its optimization plan. Additionally, he also completed the performance optimization analysis of HPCG on the CPU.



Long Jiang is skilled in mainstream programming languages, and has strong abilities in algorithm design and analysis. He has a deep understanding of MPI and OpenMP parallel programming models, and has a preliminary understanding of the CUDA C programming model. He is familiar with the Linux operating system and is proficient in using various Linux commands and uses WSL2 as his daily development platform. He is also familiar with machine learning algorithm. In this competition, he takes charge of the UnifoLM-WMA-0 optimization, employing various optimization methods to achieve acceleration, such as I/O optimization, cache optimization, and operator optimization, etc. In addition, he was also responsible for the CPU test optimization of HPCG.



Muyuan Chen has a passion for computers and possesses strong learning abilities. He is proficient in using Linux and Git, and is well-versed in C/C++ and Java programming languages. He has a strong ability in algorithm design and implementation, and can quickly adapt to different programming environments. He is familiar with technologies such as model inference security and security protection. In this competition, he mainly took charge of the HPCG GPU optimization part and the analysis and design of the UnifoLM-WMA-0 optimization.



Tao Li has a solid foundation in computer science and a strong spirit of exploration. He is proficient in Linux and Git, skilled in C and C++ programming, familiar with MPI and OpenMP parallel programming, and has a certain foundation in AI. In this competition, he mainly took charge of the design and analysis of the HPC system, and participated in the analysis and design of UnifoLM-WMA-0.

2.3. **Team slogan.**

Code Flows, Nodes Ignite, Victory Follows.

3. Design of our HPC System

This section introduces the design of the AI and HPC cluster system. It begins with an overview of the HPC system architecture, followed by detailed discussions of hardware configuration, software configuration and the interconnection architecture. Then, we do a comprehensive performance analysis for the computing performance and power consumption.

3.1. **Architectural Diagram Overview.**

The HPC cluster design we proposed is shown in Figure 3. This system adopts a dual-network design with management and data separated, consisting of 4 heterogeneous computing nodes. It not only meets the requirements of the competition benchmark but also ensures the high availability of the system.

(1) **Access and Out-of-Band Management:** External users connect to the gigabit Ethernet switch via workstations and utilize the IPMI 2.0 protocol (the blue solid line in the figure) to achieve bottom-level monitoring and out-of-band management of the entire cluster hardware. It supports remote scheduling, power on/off operations, and sensor status monitoring.

(2) **Cluster scheduling logic:** Node 0 is configured as the master node ("Master Node"), and cluster management is achieved through a gigabit CAT6 link (indicated by the dotted line in the figure). Its core functions include job scheduling, task distribution, compilation environment hosting, and maintenance of the NFS shared file system.

(3) **High-speed interconnection architecture:** The system computing network adopts NDR InfiniBand interconnection (the black solid lines in the figure), and each node is connected to the NDR-IB switch through high-performance HCA cards, providing extremely high throughput bandwidth and microsecond-level latency.

(4) **Heterogeneous computing unit**: Each node is constructed based on a dual-processor server and is connected via the PCIe bus to two AMD Radeon Pro W7900D graphics cards. The entire cluster is equipped with a total of 8 high-performance GPUs. With outstanding single/double precision floating-point computing capabilities, it precisely meets the requirements of complex HPC simulations and large-scale AI application.

3.2. **System Configuration and Architecture Analysis.**

3.2.1. *Hardware Configurations.*

The complete technical specifications of the designed HPC system are systematically presented in Table 1. This high-performance computing system is a small heterogeneous computing cluster, which is a cost-effective and balanced heterogeneous workstation cluster. Although its local storage I/O speed is slightly conservative, its powerful interconnection capability and computing density enable it to fully handle the ASC supercomputing competition and complex AI research tasks. The detailed analysis is as follows. Firstly, in terms of CPU processors, each computing node is equipped with two processors recommended by the committee, namely AMD EPYC 9755 with the latest Zen 5 architecture, each with 128 cores and 512M cache. The entire cluster has a total of 1024 physical cores, and it has strong capabilities in multi-threaded tasks and instruction-level parallelism (ILP). Secondly, at the accelerator (GPU) level, the entire cluster is equipped with 8 AMD Radeon Pro W7900D workstation graphics cards, with 2 cards per node. The W7900D is based on the RDNA 3 architecture, with a huge memory capacity and high bandwidth, making it a powerful tool for AI inference and graphics rendering. In the storage and memory level, each node is configured with 768GB DDR5 memory ($32GB \times 24$), with a clock speed of up to 6400 MT/s, ensuring a large bandwidth to meet the data feeding requirements of a multi-core CPU. The

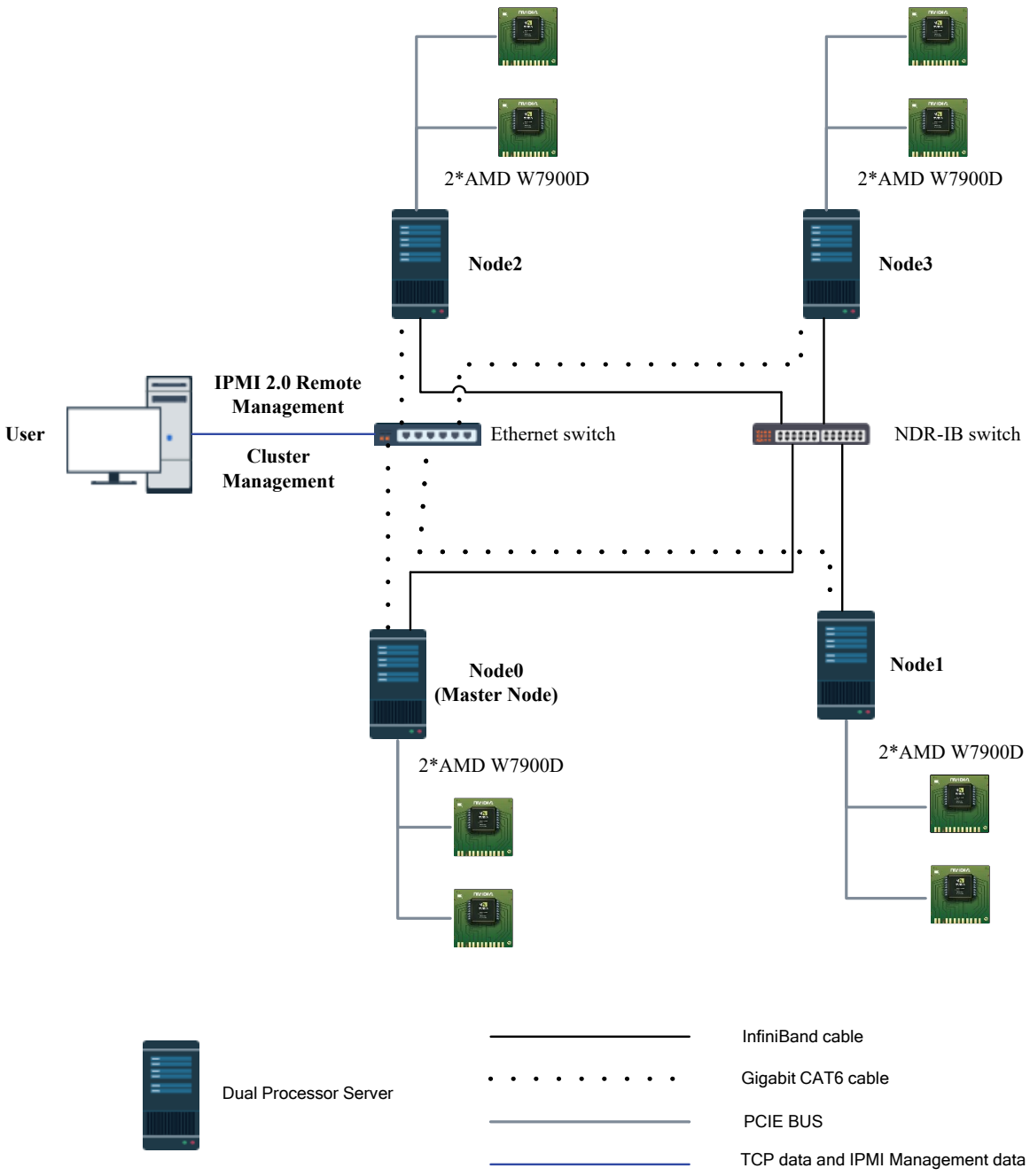


Figure 3. *Architectural Diagram Overview*

system disk uses a 960GB SATA SSD, balancing startup speed and stability. Finally, the high-speed network uses the NVIDIA Quantum-2 NDR InfiniBand switch, providing a single port throughput of 400Gbps, and combined with the MCX75310AAS-NEAT HCA card, it achieves extremely low one-way latency.

3.2.2. *Software Configurations.*

Table 1. *Hardware resources details*

Item	Name	Quantity
Dual Processor Server	CPU: AMD 9755-EPYC 2.7_128C_512M * 2	4
	Memory: 32GB * 24, DDR5, 6400 MT/s	4
	Hard disk: 960GB SSD SATA * 1	4
	Motherboard Power Consumption	4
GPU	AMD W7900D	8
HCA	NDR400: NVIDIA MCX75310AAS-NEAT	4
Switch	Ethernet switch: 10/100/1000 MB/s, 24 ports Ethernet switch	1
	NDR-IB switch: NVIDIA Quantum (TM)-2 NDR InfiniBand Switch , 64-ports NDR, 32 OSFP ports, unmanaged, P2C airflow (forward)	1
Cable	Gigabit CAT6 cables	5
	InfiniBand cable	4

Since the CPU and GPU of this HPC system are both sourced from AMD, in order to enable the system to run large-scale computing and artificial intelligence operations efficiently, its main software configuration follows the AMD recommended configuration[4]. The details are shown in Table 2. The operating system selected is Ubuntu 24.04 LTS as the underlying operating system. As the latest long-term support version, it can provide excellent stability and compatibility for high-performance hardware. For GPU heterogeneous computing, the latest stable version ROCm 7.2 is configured, providing complete support from the underlying driver to the high-level library, including MIOpen (deep learning primitive library) and rocBLAS (linear algebra library). For the CPU system deployment, Zen Software Studio 2.0 is configured, whose core includes AOCC (a compiler highly optimized for the Zen 5 architecture), AOCL (an optimized math library), ZenDNN (an inference acceleration library), and AMD uProf (a performance analysis tool). At the application layer, Anaconda is used to build an isolated and repeatable scientific computing environment, with the latest versions of PyTorch, TensorFlow, and JAX, etc. integrated.

Table 2. *Software resources details*

Item	Name	Version
Application software	Anaconda(including PyTorch, TensorFlow, JAX and so on)	Version 2.3
GPU	ROCm	Version 7.2
CPU	AMD Zen Software Studio(including AMD uProf, AOCL, AOCC, ZenDNN and so on)	Version 2.0
Operating system	Ubuntu	24.04 LTS

3.2.3. *Interconnection Architecture.*

As shown in the Figure 3, the HPC system of this design adopts a “management network + data network” separated architecture and uses a star topology. The “Master Node” serves as the cluster

[4]https://www.amd.com/en/developer.html

entry point and is mainly responsible for task distribution management. The compute nodes are mainly responsible for computing. Low-latency data exchange is achieved through InfiniBand, and remote IPMI and cluster control are realized through Ethernet.

3.3. **Computing Performance Description.**

Based on the relevant parameters of AMD 9755-EPYC [5] and AMD Radeon Pro W7900D[6], the performance analysis of the CPU and GPU for our HPC system is as follows.

The peak performance calculation formula for the CPU is

$$\text{Peak Performance} = \text{Clock Speed} \times \text{Cores} \times \text{Operations per Cycle}. \tag{3.1}$$

(1) **Clock speed:** Since the "Base clock" of AMD 9755-EPYC is 2.7GHz and the "Max. Boost Clock" is 4.1GHz, here we will calculate its highest computing performance. Therefore, we set the clock speed to 4.1GHz.
(2) **Cores:** HPC consists of 4 computing nodes, and each computing node contains 2 CPUs with 128 cores. Therefore, the cores included in HPC are $4\times2\times128=1024$.
(3) **Operations per Cycle:** In the competition, if floating-point operations use single-precision floating-point numbers, then since the Zen 5 architecture has a complete 512-bit (512-bit) vector unit, each 512-bit register can store 8 FP64 numbers. Also, since the CPU supports FMA (Fused Multiply-Add) instructions (2 FMA units), one cycle can complete one multiplication and one addition (counted as 2 operations). Therefore, the single-cycle operation count of AMD 9755-EPYC can be $2\times8\times2=32 flops/Cycle$.

Based on the above information, the final peak CPU performance of the HPC is

$$4.1GHz\times1024(Cores)\times32(\text{Operations per Cycle}) = 134.3488TLOPs \tag{3.2}$$

According to the official website information, the peak double float precision(FP64), single float precision (FP32 Vector) and half float precision(FP16) performance of the AMD Radeon Pro W7900D single GPU are 1.916 Tflops, 61.3 Tflops and 123 Tflops, respectively. And there are a total of 8 GPUs in the cluster. Therefore, the overall theoretical peak performance of the GPUs is based on different precisions are as follows.

$$\text{FP64 Peak Performance}=1.916TLOPs \times 8=15.328TLOPs. \tag{3.3}$$

$$\text{FP32 Peak Performance}=61.3TLOPs \times 8=490.4TLOPs. \tag{3.4}$$

$$\text{FP16 Peak Performance}=123TLOPs \times 8=984TLOPs. \tag{3.5}$$

3.4. **Power Consumption Description.**

Our HPC cluster comprises four computing nodes. This section provides a detailed analysis of power consumption patterns for both GPU and CPU operations at individual node level and cluster-wide scale, as detailed in Table 3. As can be seen from the table, for a single node, the CPU computing power consumption is $1054W+40W=1094W$, and the GPU computing power consumption is $354W+2\times295W=944W$, both of which meet the power consumption requirement of less than $2000W$. For the entire cluster, the CPU computing power consumption is $4926W$,

[5]https://www.amd.com/en/products/processors/server/epyc/9005-series/amd-epyc-9755.html
[6]https://www.amd.com/en/products/graphics/workstations/radeon-pro/w7900.html

and the GPU computing power consumption is $4326W$, which also meet the power consumption requirement of less than $5000W$.

Table 3. *Power Consumption Details*

Mode	Name	Power Consumption	
CPU computing	Dual Processor Server	CPU: AMD 9755-EPYC2.7 128C 512M * 2 (450w * 2)	1054w*4
		Memory: 32GB * 24, DDR5, 6400 MT/s (5w * 24)	
		Hard disk: 960GB SSD SATA * 1 (4w * 1)	
		Motherboard Power Consumption (30w)	
	GPU	AMD W7900D	20w*8
	HCA	NDR400: NVIDIA MCX75310AAS-NEAT	30w*4
	Switch	Ethernet switch: 10/100/1000 MB/s, 24 ports Ethernet switch	30w
		NDR-IB switch: NVIDIA Quantum (TM)-2 NDR InfiniBand Switch, 64-ports	400w
	Total		**4926w**
GPU computing	Dual Processor Server	CPU: AMD 9755-EPYC2.7 128C 512M * 2 (100w * 2)	354w*4
		Memory: 32GB * 24, DDR5, 6400 MT/s (5w * 24)	
		Hard disk: 960GB SSD SATA * 1 (4w * 1)	
		Motherboard Power Consumption (30w)	
	GPU	AMD W7900D (295w)	295w*8
	HCA	NDR400: NVIDIA MCX75310AAS-NEAT	30w*4
	Switch	Ethernet switch: 10/100/1000 MB/s, 24 ports Ethernet switch	30w
		NDR-IB switch: NVIDIA Quantum (TM)-2 NDR InfiniBand Switch, 64-ports	400w
	Total		**4326w**

4. HPL Test and Optimization

This section presents the analysis and optimization test results of our HPL.

4.1. **Analysis of HPL.**

The High-Performance Linpack (HPL) Benchmark serves as a standardized computational tool for evaluating the floating-point performance capabilities of HPC systems. By solving large-scale systems of linear equations **Ax**=**b**, i.e., Eq.4.1, where matrix **A** represents an $n \times n$large-scale dense matrix, this benchmark quantifies system performance in FLOPS. This method is one of the internationally recognized standards for assessing supercomputer performance.

One way to solve Eq.4.1 is Gaussian elimination, which first results in a triangular matrix by first converting the **A** in the **Ax**=**b** equation into a triangular matrix by elimination.

Suppose the solution problem is

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \tag{4.1}$$

The basic idea of solving the equation about **x** is as follows. First, the matrix transformation of **A** is carried out to obtain the following form:

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \tag{4.2}$$

Then, decompose **A**

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ \frac{a_{21}}{a_{11}} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \frac{a_{n1}}{a_{11}} & \frac{a_{n2}}{a_{22}} & \cdots & 1 \end{bmatrix} \mathbf{U} \tag{4.3}$$

where **U** is the upper triangular matrix obtained in Eq.4.2 above.

The system of equations **Ax**=**b** is converted to **LUx**=**b**, so that **y**=**Ux** then **Ly**=**b**, you can find the value of **y**, and from **Ux**=**y**, you can get the value of **x**.

The analytical derivation presented above reveals that the core computational framework of HPL fundamentally relies on matrix transformation principles and essential linear algebra operations, particularly matrix multiplication and inversion. Therefore, it is clearly that the optimization methods for HPL test should focus on the optimization of matrix operations.

4.2. **Performance Optimization Methods.**

4.2.1. *Modification of "HPL.dat".*

In the data document HPL.dat of HPL, many parameters affect the test results of HPL, among which the most important parameters are the problem size N, the block size NB, and the process array configuration ($P \times Q$). Their optimal values are usually determined by factors such as the structure of the CPU, and we can obtain their optimal values for HPL through multiple estimates

and tests. The other parameters have a smaller impact on the results, and we can also determine their optimal values through partial fine-tuning.

(1) **The size of the matrix Ns:** The N value determines the size of the matrix. And the larger the scale N of the matrix, the higher the proportion of effective computation and the higher floating-point processing performance of the system. However, at the same time, the increase in matrix scale N will lead to an increase in memory consumption. Once the actual memory space of the system is insufficient, using the cache will significantly reduce performance. Therefore, for general systems, it is necessary to try to increase the scale N of the matrix while ensuring that the system cache is not used. Considering that the operating system itself requires a certain amount of memory, in addition to matrix A ($N \times N$), HPL also has other memory overheads, and communication also needs to occupy some cache (the specific size of which varies depending on different MPIs). Generally, matrix A occupying about 80% of the total system memory is optimal $N \times N \times 8$=total system memory 80%.

(2) **The size of the matrix block(NBs):** NB represents the size of matrix block division. The size of the calculation granularity NB affects the pros and cons of the calculation performance heavily. The coefficient matrix is divided into cyclic blocks of $NB \times NB$ and assigned to each process Medium to medium processing, too coarse or too fine particles will affect performance. Because the communication between cluster nodes adopts the message passing method, the NB is too small to pass messages frequently between nodes, resulting in long communication time, short computing time, and low computing/communication performance ratio; The NB is too large, which can easily lead to uneven load, which will also increase communication overhead. Therefore, communication between and within nodes also requires trade-offs. The selection of NB value is mainly to obtain the optimal value through actual testing. According to the test, NB should not be too large or too small, generally below 512, generally a multiple of 64.

(3) **Two-dimensional processor grid ($P \times Q$):** $P \times Q$ represents the process grid, which is equivalent to the number of processors in the cluster. When choosing these two numbers, try to make the grid as "square-shape" as possible in shape. As stated on the HPL website, the best practice is to make the grid as close to a "squar" as possible, that is, P and Q should be approximately equal and Q should be slightly larger than P. Therefore, the two-dimensional processor grid ($P \times Q$) should best meet the following requirements:

$P \times Q$= the number of processes;

$P \times Q$= the number of GPUs = the number of processes.

One process corresponds to one GPU, and a process can spawn multiple threads to saturate the CPU.

$P \leq Q$.

Through multiple tests, the program is run with the number of threads=16, and it is found that when the values of P and Q are 4, 4 or 2, 8, the running results show better.

4.2.2. *Third-Party Library.*

There are two alternative third-party libraries: the MPI library and the BLAS library. Commonly used MPI libraries include MPICH, IMPI, and OpenMPI. BLAS libraries include CBLAS, MKL, OpenBLAS, and more. Most of the computation on the CPU is performed by the BLAS library, so the BLAS library has a relatively large impact on the computation. Therefore, it is still very important to choose the right BLAS library. Based on the paper and competition materials from previous years, we initially chose OpenBLAS and MKL as the basic linear algebra subroutines, followed by MPICH and OpenMP as the MPI libraries for testing, with the following results:

Table 4. *Comparison of different BLAS libraries and compilers for HPL test*

Configuration	Performance
OpenBLAS + Intel Compiler	345.9 Gflops
MKL + Intel Compiler	659.3 Gflops
OpenBLAS + GNU Compiler	341.2 Gflops
MKL + GNU Compiler	623.7 Gflops

As can be seen from Table 4, even after configuring with various third-party libraries, the performance is still poor. Considering that we are using an AMD processor and the execution results of the Intel series libraries are not satisfactory when used on AMD processors, we used the AOCC provided by AMD official website as the compiler and the AMD BLAS library released by it. Based on the binary files released by AMD official website for testing, the test results of HPL have significantly improved, as shown in Table 5. Utilizing AMD's libraries enables the performance test to reach 866.56 Gflops.

Table 5. *BLAS library and compiler of AMD for HPL test*

Configuration	Performance
AOCC + AOCL	869.00 Gflops

4.2.3. *Accelerate Procedures by Using Appropriate Compiler Options.*

In order to better achieve high-performance optimization, we have debugged the compilation options of the compiler(shown in Table 6), which helps to make full use of the compiler's computing power and can provide some necessary links for the calculation.

Table 6. *Compile flags*

Compile Options	Functional Description
-fomit-frame-pointer	Omit frame pointer
-O3	Enable all conventional optimizations
-funroll-loops	Enable loop unrolling optimization
-m64	Specify compiler to generate 64-bit code
-Wl,–start-group -Wl,–end-group	Used for linkers
-lpthread	Add a pthread library at link time
-lm	Include the Math Library when linking

4.2.4. *Multi-thread Acceleration.*

Based on the HPL testing algorithm, the matrix is decomposed into **LU** decomposition by dividing it into blocks using multiple processes. Each process is responsible for a matrix block, and OpenBLAS indeed supports multi-threading acceleration, and the number of threads used can be controlled through environment variables. Through reasonable thread configuration and parallel computation, OpenBLAS can significantly improve the performance of linear algebra calculations,

especially for large-scale matrix operations; at the same time, OpenMP also supports multithreading acceleration, which is particularly suitable for parallel computation on shared memory architectures. It is a parallel programming model for compiler instructions and runtime environments, allowing programs to execute tasks in parallel on multiple processor cores, thereby improving computational efficiency.

So we guess that if multi-threads is used to accelerate matrix calculations, multi-threads for computation can improve performance. Enhance scalability, optimize memory and cache access, help with load balancing, and improve running efficiency.

4.3. **Machine Performance Analysis.**

Table 7. *Hardware Configuration*

Name	Model	Quantity
CPU	AMD EPYC 9124 16-Core Processor	1
GPU	NVIDIA GeForce RTX 4070 Ti Super	2
RAM	DDR5	128G
Hard Disk	SSD	1T

The server configurations used for the HPL and HPCG tests are as shown in the Table 7. Therefore, based on the corresponding information from the official website, we have obtained the theoretical peak performance of the server's CPU and GPU as follows.

(1) **The overall CPU theoretical peak performance is:**

$$3.7GHz * 16(cores) * 16(\text{Operations per Cycle}) = 947.2Gflops \tag{4.4}$$

(2) **The theoretical peak performance of one GPU is:**

$$689Gflops(GPU) * 1 = 689Gflops \tag{4.5}$$

4.4. **Experimental Results.**

4.4.1. *Software Setup.*

The main software information required for the HPL test is shown in Table 8.

Table 8. *Software Environment Configuration*

Item	Name	Version
MPI	OpenMPI	4.1.2 version
Mathematical Libraries	AMDBlis	5.0.0 version
	AOCL	5.0.0 version
GPU compiler and runtime	CUDA Toolkit	12.6 version
CPU compiler and runtime	AOCC	5.0.0 version
Operating system	Ubuntu	22.04.2 LTS

4.4.2. *CPU Evaluation Results.*

In order to test the performance in the CPU, we tried different combinations of values of N, NB, and $P \times Q$ in the configuration file "HPL.dat". The final best parameters in "HPL.dat" are N=70000, NB=384,$P \times Q$=1×1. And the performance result is shown in Figure 4 with 869.00 Gflops. Finally, we can obtain the maximum peak efficiency is

$$\text{Efficiency}=\frac{\text{Peak of HPL}}{\text{Theoretical Peak}}=\frac{869.00\ \text{Gflops}}{947.2\ \text{Gflops}}\approx 91.74\% \tag{4.6}$$

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WRC06R8C48c 211    70000  384     1     1             263.15             8.6900e+02
HPL_pdgesv() start time Sun Feb 22 00:35:12 2026
HPL_pdgesv() end time   Sun Feb 22 00:39:36 2026
--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=   4.14212426e-03 ..... PASSED
--------------------------------------------------------------------------------
```

Figure 4. *Performance result of the CPU HPL test*

4.4.3. *GPU Evaluation Results.*

First, we configured the HPL test of the GPU based on experience and found that our test results were not good. Then considering that we are using AMD processors, we decided to learn the binary packages that AMD uses. Figure 5 shows the best profile for the performance test with N=36864, NB=1024, $P \times Q$=1×1. Finally, we can get the efficiency is

$$\text{Efficiency}=\frac{\text{Peak of HPL}}{\text{Theoretical Peak}}=\frac{632.1\ \text{Gflops}}{689\ \text{Gflops}}\approx 91.74\% \tag{4.7}$$

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops (   per GPU)
--------------------------------------------------------------------------------
WC0                36864  1024     1     1              52.84             6.321e+02 ( 6.321e+02)

HPL_pdgesv() start time Thu Feb 26 00:52:38 2026
HPL_pdgesv() end time   Thu Feb 26 00:53:30 2026

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=   0.000545708949 ..... PASSED
||Ax-b||_oo . . . . . . . . . . . . . . . . . = 0.0000000003160978
||A||_oo . . . . . . . . . . . . . . . . . . . = 9333.1952734906790283
||x||_oo . . . . . . . . . . . . . . . . . . . = 15.1640722058340796
||b||_oo . . . . . . . . . . . . . . . . . . . = 0.4999965347862580
================================================================================
```

Figure 5. *Performance result of the single GPU HPL test*

4.5. **Summary.**

To sum up, we can see that in HPL testing, in addition parameters N, NB, $P \times Q$, affecting the test results, different BLAS libraries and compilers perform differently on specific hardware. In the end, our **CPU efficiency and GPU efficiency all reached about** 91.74%.

5. HPCG Test and Optimization

This section focuses on the analysis and evaluation of HPCG. All the test results presented are derived from the same hardware platform and software configuration for HPL, as detailed in Table 7 and Table 8.

5.1. **Analysis of HPCG.**

The High Performance Conjugate Gradient (HPCG) benchmark is a key tool for assessing the performance of high-performance computing (HPC) systems, particularly in simulating real-world computational workloads. Unlike traditional benchmarks such as HPL, which mainly measure floating-point operations through dense matrix multiplication, HPCG focuses on solving large sparse linear systems. These systems are derived from the discretization of a three-dimensional partial differential equation (PDE) that simulates a heat diffusion problem with zero Dirichlet boundary conditions.

At the heart of HPCG is the preconditioned conjugate gradient (PCG) method, which necessitates efficient handling of sparse matrix operations, optimized memory bandwidth, and parallel computation across distributed systems. The global problem is defined on a three-dimensional grid with dimensions $(nx \times npx) \times (ny \times npy) \times (nz \times npz)$, where nx, ny, and nz represent the local subgrid sizes in the x, y, and z dimensions, respectively, assigned to each MPI process. The values npx, npy, and npz denote the number of processes in each corresponding dimension. These values are automatically computed during the HPCG setup phase but can also be manually specified in the input file "hpcg.dat" or provided as command-line arguments.

The HPCG benchmark consists of three critical computational kernels, each integral to evaluating modern supercomputer architectures:

(1) Sparse Matrix-Vector Multiplication (SpMV): The core computational task of HPCG, SpMV is computationally intensive and central to the benchmark's performance evaluation.
(2) Symmetric Gauss-Seidel (SymGS) Relaxation: This serves as the essential preconditioning technique for the conjugate gradient method, enhancing the efficiency of the iterative solution process.
(3) Conjugate Gradient Method: The iterative approach used to solve the large-scale sparse linear systems, making it a crucial component of HPCG's overall performance.

These kernels are commonly found in scientific computing applications, such as computational fluid dynamics and electromagnetic simulations. They present significant challenges for system bandwidth and fine-grained parallel efficiency, making them indispensable for benchmarking modern HPC architectures.

5.2. **Performance Optimization Methods.**

5.2.1. *Modification of "HPCG.dat".*

By modifying the configuration parameters in the "HPCG.dat" file, the optimization of the standard HPCG benchmark can be significantly impacted. The file contains two key sections that can be adjusted to improve performance: the problem size (specifically, the matrix size) and the runtime. Adjusting the matrix size can have a major effect on performance.

(1) **Matrix Size:** The matrix size defines the scale of the problem and is directly related to the global dimensions of the discretized computation grid. By adjusting the local subgrid dimensions (nx, ny, nz) and the number of subgrids in each dimension (npx, npy, npz), the workload assigned to each MPI process can be optimized. A larger matrix size generally increases computational complexity, which improves the scalability and performance of

the benchmark on a large number of processors. However, excessively large problem sizes may lead to memory limitations or imbalanced workload distribution between processes. Therefore, careful consideration of these parameters is essential for achieving optimal performance.

(2) **Runtime:** The runtime is another important parameter that influences the performance of the HPCG benchmark. A longer runtime allows for more thorough testing of the system's ability to handle sustained computational load. While a longer runtime provides a better measure of the system's long-term stability and computing capacity, excessively long runtimes may lead to diminishing returns in performance due to hardware bottlenecks. Therefore, choosing an appropriate runtime is essential to strike a balance between accuracy and benchmark duration. In addition to the size of the topic scale, the shortest running time should be set to ensure the long enough running time of the system. Optimizing the matrix size and runtime in the "HPCG.dat" file is crucial for fine-tuning benchmark performance, ensuring that the results reflect the system's true capabilities. These modifications help with better load distribution, reduced memory overhead, and improved overall efficiency in parallel computing environments.

5.2.2. *Third-Party Library for HPCG.*

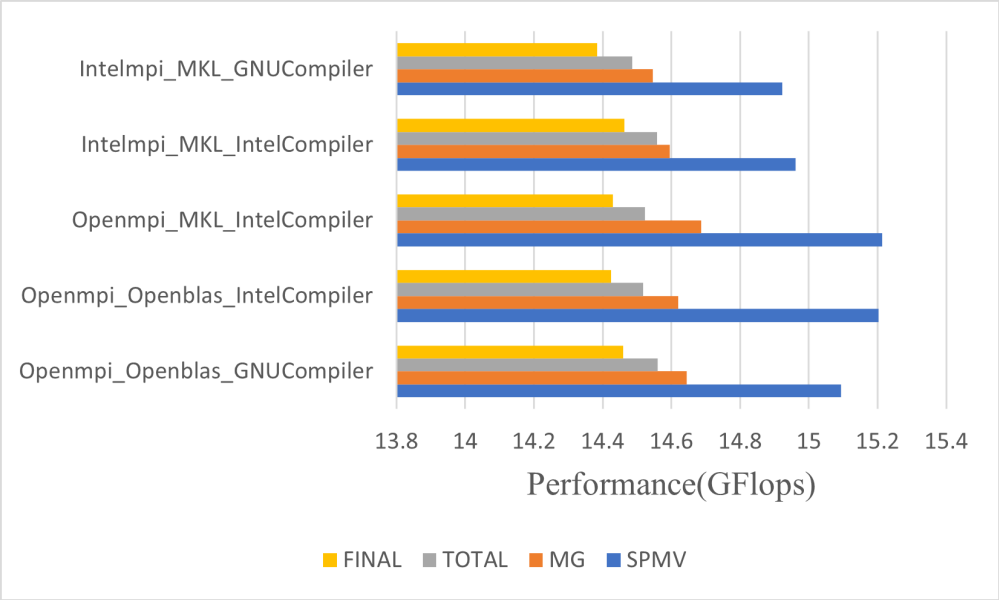


Figure 6. *Performance results for different third-party library settings in standard HPCG Benchmark*

For the standard HPCG benchmark, there are several third-party libraries available that can enhance performance. Key libraries include those for MPI and BLAS. The commonly used MPI libraries are MPICH, OpenMPI, and Intel MPI (IMPI). These libraries handle the communication between different processes in a distributed computing environment. For BLAS, several implementations are available, such as MKL (Intel Math Kernel Library), OpenBLAS, and Atlas. BLAS libraries are crucial for performing optimized linear algebra operations on the CPU, which form a significant part of the HPCG computation. After testing, we chose OpenBLAS and MKL as the

BLAS libraries, while OpenMPI and Intel MPI were selected as the MPI libraries for communication. The matrix size used in our experiments is set to $(160 \times 160 \times 128)$ and the runtime is fixed at 1800 seconds, which corresponds to the optimal parameter setting in Figure 6.

5.2.3. *Parallelization Paradigms for HPCG.*

In this subsection, we explore three different configurations for the standard HPCG benchmark: MPI alone, OpenMP alone, and both MPI and OpenMP together. These configurations allow us to compare the performance of the standard HPCG benchmark under different parallelization paradigms and understand the impact of each parallelization method. The matrix size used in our experiments is set to $(160 \times 160 \times 128)$, with the runtime fixed at 1800 seconds. The third-party libraries used are OpenMPI and OpenBLAS, and the GNU Compiler is employed for compilation.

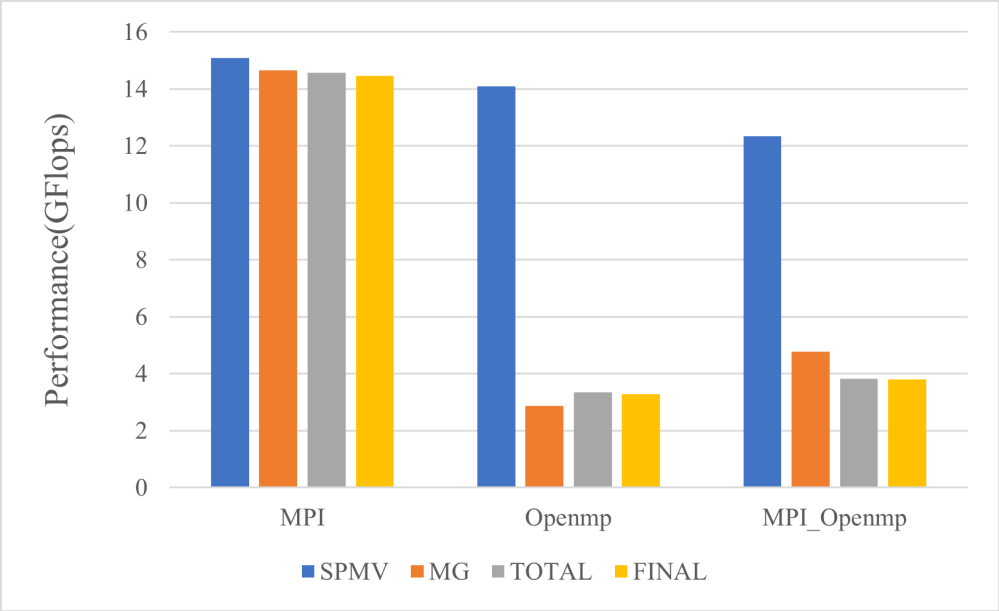


Figure 7. *Performance results for different Parallelization Paradigm settings in standard HPCG Benchmark*

Based on the results of the experiments, as shown in Figure 7, we observed that the configuration using MPI alone achieved the best performance across all tested machines. This configuration allowed the standard HPCG benchmark to fully utilize the distributed memory capabilities of the system, with each MPI process handling a portion of the problem. The communication overhead, though present, was minimized due to the high efficiency of MPI in handling large-scale parallel problems. In contrast, using OpenMP alone resulted in lower performance, primarily due to the limited scalability on a single-node setup. Although OpenMP effectively leveraged multicore CPUs, it was constrained by memory bandwidth and the relatively limited computational power available within a single node. The hybrid configuration performed well on multi-node systems, but the performance improvement was not as significant as expected. This is likely due to suboptimal load balancing between MPI and OpenMP, where the communication overhead between processes overshadowed the benefits of multi-threading within nodes. Moreover, hybrid configurations often require fine-tuning to achieve maximum performance, especially with larger problem sizes, which can result in a less straightforward optimization process.

5.2.4. *Hybrid Version of HPCG.*

To further optimize performance, a hybrid approach can be applied to the HPCG benchmark, integrating CUDA for GPU acceleration. This allows the GPU's floating-point performance to be exploited without altering the basic conjugate gradient (CG) algorithm or preconditioner algorithm. The GPU's parallel processing power is harnessed to significantly speed up matrix operations, which are a critical part of the HPCG benchmark. In this hybrid version, the matrix is analyzed, reordered, and the computational workload is split between the CPU and GPU. The CPU handles portions of the matrix that are better suited for traditional processors, while the GPU is responsible for the portions that benefit most from parallel computation capabilities. This approach allows the hybrid version to capitalize on the strengths of both the CPU and GPU, optimizing performance for large-scale computations. Fine-tuning the ratio of CPU to GPU workload is essential to achieve optimal performance. By experimenting with different configurations and workloads, the ideal distribution of tasks between CPU and GPU can be found, maximizing the overall performance of the HPCG benchmark on systems equipped with both CPU and GPU resources. The hybrid version allows HPCG to fully utilize both CPU and GPU resources, providing an effective solution for high-performance computing systems with multi-core CPUs and GPUs.

5.3. **Experimental Results.**

5.3.1. *CPU Evaluation Results.*

Since our server uses an AMD CPU, using the AMD-specific HPCG benchmark allows for the full utilization of its hardware features, resulting in a more accurate performance evaluation. Compared to the standard HPCG benchmark, the AMD version is optimized to better reflect performance in real-world applications. The following describes the test experiments for the AMD-specific HPCG benchmark. First, We conducted experiments by using only the CPU to run the AMD-specific HPCG benchmark. To achieve the best performance, we tested different data scales by adjusting the matrix size in the configuration file "HPCG.dat". By fine-tuning the matrix size, we sought to identify the optimal configuration for CPU-based computations on our AMD hardware. And finally we get the best performance 21.2034 GFlops with:

- **Matrix Size:** $512\times256\times256$.
- **Number of subgrids:** npx=4, npy=2, npz=2.
- **runtime:** 1800s.

So, the efficiency of the CPU evaluation is

$$\text{Efficiency}=\frac{\text{Peak of HPL}}{\text{Theoretical Peak}}=\frac{\text{21.2034 GFlops}}{\text{947.2 GFlops}}\approx 2.24\% \tag{5.1}$$

5.3.2. *GPU Evaluation Results.*

For GPU evaluation, we utilized the CUDA-enabled version of the HPCG benchmark. The version we used is provided by NVIDIA and cannot be modified. Although we attempted to analyze the source code as time permitted, it is undeniable that there is a significant gap between our understanding and the optimizations implemented in the NVIDIA-provided version. Therefore, our current optimizations for HPCG were primarily achieved through parameter tuning in the "HPCG.dat" file. These adjustments contributed to improved performance on GPU-based computations. After testing multiple configurations, the optimal performance achieved was 232.673 GFlops with

- **Matrix Size:** $256\times256\times256$.
- **Number of subgrids:** npx=1, npy=1, npz=2.

- **runtime:** 1800s.
- **Number of GPUs:** 2 (Dual GPU configuration).

$$\text{Efficiency} = \frac{\text{Peak of HPL}}{\text{Theoretical Peak}} = \frac{232.673 \text{ GFlops}}{689 \text{ GFlops} \times 2} \approx 16.88\% \tag{5.2}$$

5.4. **Summary.**

In the HPCG tests, our **CPU efficiency reached 21.2034 GFlops with about 2.24% efficiency**, and **the GPU's computational peak reaches 232.673 GFlops with about 16.88% efficiency**.

6. Optimization for UnifoLM-WMA-0 Inference

6.1. **Introduction.**

UnifoLM-WMA, developed by Unitree Robotics, is an embodied artificial intelligence foundation model that deeply integrates a generative world model with an action policy model [3]. By leveraging a "mental rehearsal" mechanism to simulate physical consequences, it equips robots with high-fidelity environmental prediction and complex task decision-making capabilities. However, practical deployment faces a critical computational bottleneck: the massive inference load driven by high-dimensional continuous video synthesis and intensive iterative state denoising severely restricts high-frequency real-time control and long-horizon planning.

In this competition, our primary objective is to accelerate the inference speed of UnifoLM-WMA while strictly maintaining a Peak Signal-to-Noise Ratio (PSNR) above 25(compared to the committee's reference videos) across all test cases. To achieve this, our team explored five distinct optimization strategies: NUMA Affinity Optimization, Mixed Precision, I/O Improvements, FlashAttention Recomputation, and DeepCache Implementation. A detailed discussion of these methods is provided in Section 6.2.

Our empirical evaluation is conducted across two hardware environments (detailed in Section 6.3.1), named as "Server A" and "Server B". "Server B" was utilized during our initial exploration phase; although its PSNR performance of the baseline test did not meet the committee's formal requirements, the data gathered from it is instrumental in analyzing the specific trade-offs and efficacies of each optimization technique. "Server A" serves as our definitive submission platform for validating the final results.

To ensure robust compliance with the competition's rigorous standards, we ultimately deployed a synergistic optimization pipeline on "Server A", integrating Mixed Precision, I/O Improvements, and FlashAttention Recomputation. The comprehensive performance results of this final solution are presented in Section 6.3.2.

To quantitatively assess the impact of these optimization strategies on inference performance, we conducted baseline and optimized evaluations on both "Server A" and "Server B", with statistical comparisons of PSNR and inference time. The baseline environment was strictly configured following the official guidelines provided by the organizing committee and the Unitree code repository.

On "Server A", we analyzed 20 test cases. The baseline averaged 39.72 dB in PSNR and 578.2 seconds in inference time. After optimization, the average inference time dropped to 404.5 seconds, yielding a mean speedup of 1.407×, while the post-optimization PSNR remained above 39 dB, showing negligible fluctuation from the baseline. All individual cases maintained a PSNR above the required 25 dB threshold.

On "Server B", although the baseline PSNR averaged only 21.43 dB (below the 25 dB requirement), the PSNR values remained virtually unchanged after optimization, averaging approximately 21.4 dB. Concurrently, the inference time was reduced from 1296.1 seconds to 336.9 seconds, achieving an average speedup of 3.822×. Given the consistent PSNR, the speedup measurements obtained on "Server B" retain meaningful comparative value for assessing the relative effectiveness of different optimization methods.

6.2. **Optimization Methods.**

To systematically enhance the inference efficiency of UnifoLM-WMA, we designed a collaborative optimization scheme across three levels, as illustrated in Figure 8.

(1) System-Level Pre-configuration

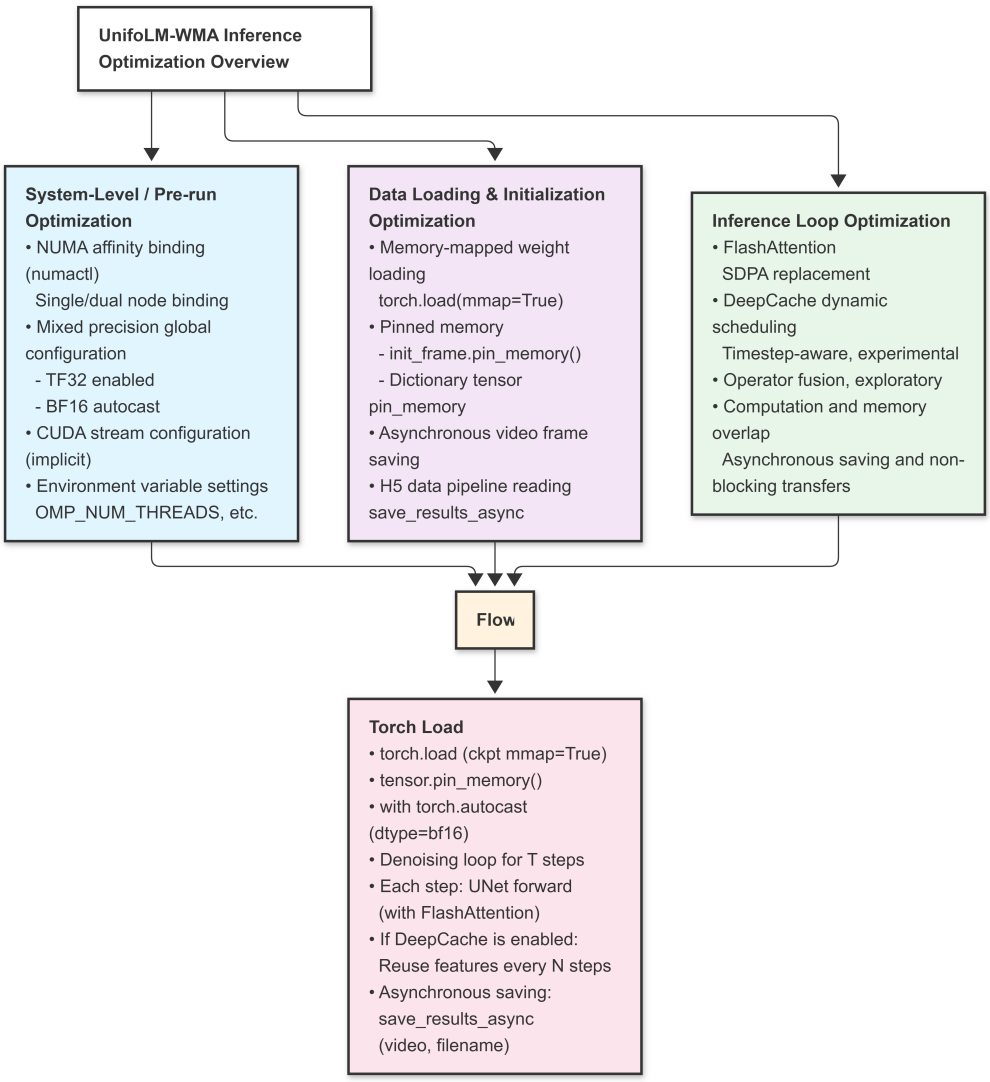


Figure 8. *Workflow Optimization Map*

Prior to model execution, NUMA affinity is enforced using the numactl tool to ensure that CPU memory accesses and GPU resources are confined to the same physical node, thereby eliminating latency overhead incurred by cross-node communication. Concurrently, global mixed-precision settings are enabled: TF32 format is permitted to accelerate matrix operations, while BF16 automatic mixed precision (autocast) is applied in core computational regions to reduce memory footprint and data transfer volume.

(2) Data Loading and I/O Optimization

To address bottlenecks associated with loading large-scale model weights and input data, we employ memory-mapped loading via torch.load(mmap=True), eliminating redundant CPU memory copies. Input images and tensors are pinned to page-locked memory using .pin memory() to accelerate CPU-to-GPU transfer efficiency. For H5-format input data, a pipelined reading mechanism is constructed to minimize I/O waiting. For video frame saving, an asynchronous saving function save results async is designed to offload post-processing and disk write operations to background threads, allowing the main thread to proceed directly to the next inference iteration.

(3) Inference Loop Optimization

Within the core denoising loop, FlashAttention replaces standard attention implementations to reduce memory read/write overhead. The DeepCache scheduling strategy remains under experimental investigation to determine optimal parameter configurations: through a timestep-aware caching mechanism, deep features are reused every N steps, bypassing full U-Net forward passes and thereby reducing computational load. Computation and storage overlap is achieved through asynchronous saving and non-blocking transfers, ensuring sustained GPU computation during I/O operations. Operator fusion is currently in the exploratory phase and is considered a direction for future optimization.

Overall Data Flow: After model weights are loaded via memory mapping, input data is transferred to the GPU through pinned memory. Within the BF16 automatic mixed-precision context, a T-step denoising loop is executed, where each U-Net forward pass integrates FlashAttention; if DeepCache is enabled, features are reused at set intervals. The generated video is ultimately saved to disk via the asynchronous function, achieving substantial overlap between computation and I/O.

6.2.1. *NUMA Affinity Optimization.*

Table 9. *Impact of NUMA Affinity Optimization on Server B Inference,where the baseline performance is detailed in Section 6.3.2. And, the values in the parentheses of the "Test Time" and "Test PSNR" columns respectively represent the speedup ratio relative to the baseline time and the difference to baseline PSNR.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	19m26.800s / 1166.800 s (1.230×)	18.5229(0)
	2	19m25.073s / 1165.073 s(1.236×)	18.5432(0)
	3	19m20.739s / 1160.739 s(1.241×)	19.9578(0)
	4	19m22.388s / 1162.388 s(1.236×)	17.7746(0)
unitree_z1_stackbox	1	20m54.452s / 1254.452 s(1.238 ×)	23.9826(0)
	2	20m53.071s / 1253.071 s(1.243×)	20.0572(0)
	3	20m53.853s / 1253.853 s(1.240×)	21.7059(0)
	4	20m52.502s / 1252.502 s(1.243 ×)	25.2775(0)
unitree_z1_dual_arm_stackbox_v2	1	19m33.918s / 1173.918 s(1.234×)	20.0181(0)
	2	19m29.337s / 1169.337 s(1.238×)	18.8915(0)
	3	19m32.557s / 1172.557 s(1.236×)	18.8271(0)
	4	19m31.043s / 1171.043 s(1.237 ×)	19.3431(0)
unitree_z1_dual_arm_stackbox	1	13m5.162s / 785.162 s (1.226×)	22.8169(0)
	2	13m5.668s / 785.668 s (1.229×)	24.5789(0)
	3	13m5.123s / 785.123 s (1.225×)	26.2675(0)
	4	13m4.947s / 784.947 s (1.228×)	25.3952(0)
unitree_z1_dual_arm_cleanup_pencils	1	14m31.863s / 871.863 s(1.231×)	21.2321(0)
	2	14m32.574s / 872.574 s(1.234×)	22.5411(0)
	3	14m33.037s / 873.037 s(1.229×)	19.9960(0)
cline2-4	4	14m32.806s / 872.806 s(1.234×)	22.8648(0)
Average	-	17m45.346s / 1065.346s(1.234×)	21.4297(0)

From the perspective of Non-Uniform Memory Access (NUMA) topology, "Server A" features a single-node structure, whereas "Server B" employs a dual-node architecture. The optimization strategy detailed herein is specifically designed for multi-node NUMA systems, as exemplified by "Server B".

In a dual-socket NUMA configuration, CPUs, memory, and GPUs are physically partitioned across distinct hardware nodes (sockets). Consequently, when a computational process residing on one node attempts to access memory or GPU resources located on the other, the data must traverse the interconnect bus (e.g., QuickPath Interconnect or UltraPath Interconnect). This cross-node communication introduces substantial latency and is strictly constrained by the bandwidth limits of the bus. During intensive, large-scale inference tasks, such architectural constraints frequently induce data congestion, manifesting as a critical performance bottleneck.

To mitigate this bottleneck, we utilize the numactl utility to enforce strict hardware locality. By leveraging OS-level scheduling, this approach confines script execution, CPU scheduling, memory allocation, and target GPU utilization entirely within a single physical node. This strategy guarantees a precise topological alignment between computational tasks and their requisite hardware resources. By circumventing the redundant overhead associated with cross-node communication, this method substantially enhances the overall system throughput and accelerates model inference, **resulting in an approximately 23.4% reduction in inference time as shown in Table 9(the baseline test on "Server B" is shown in Table 18).** Since NUMA binding operates strictly as an OS-level optimization for hardware resource scheduling, it does not interfere with the model's underlying mathematical logic. **Consequently, the PSNR of the generated videos remains unaffected, yielding a mean variance of zero.**

6.2.2. *Mixed Precision.*

The fundamental principle of mixed-precision optimization relies on the fact that deep neural networks are generally insensitive to minor numerical variances, allowing them to maintain output accuracy without requiring extremely high decimal precision. During model inference, replacing standard 32-bit floating-point (FP32) formats with 16-bit BF16 or TF32 (TensorFloat-32) formats directly halves the total data volume of model parameters and intermediate computations. This reduction significantly alleviates the data transfer payload between system memory and the GPU, effectively mitigating I/O bottlenecks and accelerating transmission speeds. Specifically, BF16 retains the same exponent width as FP32 by truncating the mantissa, thereby preserving the dynamic range; meanwhile, TF32 leverages hardware-level advantages for low bit-width multiplication during internal computations. Furthermore, because the individual data elements are shorter, modern GPU computing units can process a greater number of floating-point operations in parallel within a single clock cycle, multiplicatively increasing the throughput of existing hardware. In summary, mixed precision trades unnecessary decimal precision for a smaller memory footprint and faster data flow, tangibly improving overall inference speed while ensuring minimal degradation in the generated video quality (PSNR).

Regarding specific implementation, this optimization can be divided into three independent optional components.

(1) First, at the global backend configuration level, we enable hardware-level TF32 acceleration **torch.backends.cuda.matmul.allow_tf32 = True** and **torch.backends.cudnn.allow_tf32 = True**. These directives allow the CUDA and cuDNN core libraries to automatically convert input FP32 data into the TF32 format during matrix multiplications and convolutions. The TF32 format utilizes an 8-bit exponent to maintain the same dynamic range as FP32 to prevent overflow, while retaining a 10-bit mantissa to satisfy the model's precision requirements.

(2) Second, we introduce a BF16-based Automatic Mixed Precision (AMP) inference architecture within the core diffusion denoising loops, such as action sequence vector generation and video sequence frame generation. By utilizing the **torch.autocast(device_type"cuda",**

Table 10. *Impact of Mixed Precision Optimization on Server A Inference, where the baseline performance is detailed in Section 6.3.2.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	9m29.502s / 569.502 s (1.297×)	36.2602(0.619)
	2	7m43.190s / 463.190 s (1.357×)	42.7647(9.462)
	3	7m50.046s / 470.046 s (1.333×)	34.4601(-4.614)
	4	7m41.468s / 461.468 s (1.365×)	33.5926(-0.535)
unitree_z1_stackbox	1	8m13.019s / 493.019 s (1.387×)	48.3723(-0.542)
	2	8m15.012s / 495.012 s (1.377×)	47.5901(6.848)
	3	8m14.072s / 494.072 s (1.363×)	48.0674(-1.713)
	4	8m18.593s / 498.593 s (1.364×)	47.1862(-0.075)
unitree_z1_dual_arm_stackbox_v2	1	7m48.821s / 468.821 s (1.355×)	25.2486(-0.819)
	2	7m45.663s / 465.663 s (1.365×)	27.1028(-3.405)
	3	8m3.851s / 483.851 s (1.326×)	32.4244(0.431)
	4	7m52.203s / 472.203 s (1.351×)	34.2318(-4.998)
unitree_z1_dual_arm_stackbox	1	5m32.981s / 332.981 s (1.307×)	43.8239(4.693)
	2	5m32.647s / 332.647 s (1.307×)	42.1073(2.184)
	3	5m33.863s / 333.863 s (1.312×)	36.0992(-0.586)
	4	5m33.525s / 333.525 s (1.308×)	47.0527(5.542)
unitree_z1_dual_arm_cleanup_pencils	1	5m57.565s / 357.565 s (1.348×)	45.5420(2.243)
	2	5m59.465s / 359.465 s (1.344×)	47.2762(-1.403)
	3	5m58.024s / 358.024 s (1.345×)	47.1712(5.012)
	4	5m59.335s / 359.335 s (1.334×)	46.5297(0.124)
Average	-	7m10.142s / 430.142 s (1.342×)	40.6452 (0.9232)

dtype=torch.bfloat16) context manager, we establish an intelligent precision scheduling mechanism. This mechanism dynamically identifies operator characteristics within the computational graph: inputs for compute-intensive operators (e.g., Linear, Conv2d) are automatically converted to BF16 and routed to Tensor Cores for rapid computation, whereas precision-sensitive operators (e.g., LayerNorm, Softmax) are strictly preserved in FP32 to ensure numerical stability.

(3) Third, we perform precision truncation on the weights of the **unifolm_wma_dual.ckpt** and **ViT-H-14** models before transferring them from CPU memory to the GPU. This strategy effectively halves the data transmission pressure on the PCIe(Peripheral Component Interconnect Express) bus during model weight loading.

On Server A, we initially attempted to apply all three mixed-precision measures simultaneously. However, we observed that the PSNR values for certain test cases failed to meet the required threshold. Consequently, we adopted a conservative mixed-precision optimization strategy for Server A, implementing only the first two measures and explicitly disabling AMP within the world model's video sequence frame generation loop. **This optimization delivered an average per-case speedup of 1.342×.** The specific optimization results are as shown in Table 10.

For Server B, we employed an aggressive mixed-precision strategy, applying all three optimization measures simultaneously. **The specific results are detailed in Table 11 with an average per-case speedup of 2.145×.**

Table 11. *Impact of Mixed Precision Optimization improved on Server B Inference, where the baseline performance is detailed in Section 6.3.2.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	10m58.948s / 658.948 s (2.178×)	18.707(0.180)
	2	10m58.109s / 658.109 s (2.189×)	18.708(0.170)
	3	10m57.763s / 657.763 s (2.189×)	19.662(0.300)
	4	10m58.393s / 658.393 s (2.182×)	17.512(-0.260)
unitree_z1_stackbox	1	11m45.120s / 705.120 s (2.203×)	24.146(0.160)
	2	11m46.028s / 706.028 s (2.205×)	20.153(0.100)
	3	11m45.068s / 705.068 s (2.205×)	21.766(0.060)
	4	11m41.959s / 701.959 s (2.218×)	23.450(-1.830)
unitree_z1_dual_arm_stackbox_v2	1	11m8.558s / 668.558 s (2.166×)	19.174(-0.840)
	2	11m7.047s / 667.047 s (2.171×)	19.730(0.840)
	3	11m9.955s / 669.955 s (2.164×)	19.836(1.010)
	4	11m13.538s / 673.538 s (2.150×)	19.515(0.170)
unitree_z1_dual_arm_stackbox	1	7m50.265s / 470.265 s (2.047×)	23.716(0.900)
	2	7m45.591s / 465.591 s (2.074×)	25.844(1.270)
	3	7m45.791s / 465.791 s (2.065×)	25.235(1.030)
	4	7m50.022s / 470.022 s (2.051×)	25.368(0.030)
unitree_z1_dual_arm_cleanup_pencils	1	8m27.712s / 507.712 s (2.114×)	20.366(-0.870)
	2	8m32.725s / 512.725 s (2.100×)	20.033(-2.510)
	3	8m28.134s / 508.134 s (2.111×)	20.235(0.240)
	4	8m26.754s / 506.754 s (2.125×)	22.741(0.120)
Average	-	10m1.874s / 601.874 s (2.145×)	21.295 (-0.1327)

As mixed precision is adopted, where the numerical precision of bf16 and tf32 is lower than that of float32 which is used by unifolm-wma as the original precision, inferences with mixed precision on Both Server A and Server B lead to a decrease in PSNR values for some cases.

6.2.3. *I/O Improvements.*

We employed three distinct I/O optimization methods, which are described as follows.

(1) Mmap-Based Loading Optimization for Massive Model Weights

In the I/O optimization phase, we systematically refactored the loading mechanism for the massive 16.7 GB model checkpoint (unifolm_wma_dual.ckpt). Specifically, we enabled memory mapping by setting **mmap=True** within the **torch.load** function, fundamentally streamlining the underlying data flow.

Under the default loading paradigm, data must traverse a strictly serial path: SSD → OS Kernel Buffer → User Space Memory → GPU VRAM(Video Random Access Memory). By **implementing memory mapping**, the file data is loaded exclusively into the kernel buffer. PyTorch can then access this data directly, circumventing redundant user-space copying and eliminating the substantial latency overhead associated with duplicating approximately **16 GB** of data within the host memory.

Crucially, the **mmap** mechanism **facilitates asynchronous, on-demand data retrieval**. When executing subsequent operations to physically migrate the weights to the GPU (e.g., invoking model.cuda()), the system triggers **page faults** to fetch the actual data from the SSD in standard page increments (typically 4KB or 2MB).

This behavior allows the PCIe bus to perform concurrent data transfers—moving data from the SSD to the kernel buffer while simultaneously transferring loaded pages from host memory to the GPU VRAM—thereby establishing a highly efficient **pipelined I/O architecture.** Compared to traditional blocking I/O, which leaves bus bandwidth underutilized (as the SSD-to-CPU transfer must complete entirely before the CPU-to-GPU transfer begins), this strategy maximizes the hardware throughput of the entire I/O pipeline, significantly reducing the end-to-end model initialization latency.

(2)I/O Transfer Optimization via Page-Locked Memory

We utilized the **page-locked memory** mechanism—specifically, the **.pin_memory()** method for PyTorch tensors—to optimize the I/O transfer paths for input data such as .h5 files and initial frame images, thereby reducing the overhead of redundant host-to-host memory copies. The efficacy of this optimization lies in its ability to eliminate implicit data copying.

In conventional heterogeneous transfer architectures relying on pageable memory, the GPU cannot safely execute Direct Memory Access (DMA) due to the dynamic paging of physical memory addresses. Consequently, the CUDA driver must implicitly allocate a temporary pinned staging buffer on the host. The CPU first copies the data from the pageable region to this staging buffer, after which the GPU's DMA engine pulls the data across the PCIe bus into the VRAM. This "double-copy" mechanism substantially exacerbates CPU scheduling overhead, host memory bandwidth consumption, and overall transfer latency.

By adopting the **pinned memory** optimization paradigm, the physical addresses of the target memory pages are strictly locked into residency by the operating system. This allows the system to completely bypass the intermediate staging phase. The CUDA driver directly authorizes the GPU's DMA controller to access and read the host's physical memory via the PCIe bus, effectively eliminating the auxiliary CPU-to-CPU memory copy. However, it should be noted that because the inherent data volume of these specific input files is **relatively small**, the tangible performance gains derived from this particular I/O optimization remain marginal.

(3)Asynchronous Non-Blocking Optimization for Video Frame Storage

Subsequently, we implemented a **non-blocking, asynchronous optimization pipeline** for the entire lifecycle of video sequence frames, from generation to storage. This necessitated refactoring the **save_results** function within the *unifolm-world-model-action/scripts/evaluation/world_model_interaction.py* script, replacing it entirely with our custom **save_results_async** function.

```
def save_results_async(video_tensor: Tensor, filename: str, fps: int =
    8) -> None:
    """Async IO optimized version of save_results."""

    # 1. All math operations, pixel clamping, and type conversions are
       done quickly on the GPU.
    with torch.no_grad():
        video = torch.clamp(video_tensor.detach().float(), -1., 1.)
        n = video.shape[0]
        video = video.permute(2, 0, 1, 3, 4)

        frame_grids = [
            torchvision.utils.make_grid(framesheet, nrow=int(n),
               padding=0)
            for framesheet in video
        ]
```

```
        grid = torch.stack(frame_grids, dim=0)
        grid = (grid + 1.0) / 2.0

        # Convert directly to uint8 (1 byte) on the GPU, instantly reducing the
        # massive fp32 (4 bytes) tensor volume by 75%.
        grid = (grid * 255).to(torch.uint8).permute(0, 2, 3, 1)

        # 2. Transfer the lightweight uint8 data (compressed by 75%) to CPU memory at once.
        grid_cpu = grid.cpu()

    # 3. Submit to background thread pool for H264 encoding and disk writing
    # (main thread returns immediately, letting GPU compute the next step).
    io_pool.submit(
        torchvision.io.write_video,
        filename,
        grid_cpu,
        fps,
        video_codec='h264',
        options={'crf': '10'}
    )
```

The rationale for this optimization addresses the severe bottlenecks inherent in the baseline data flow. Originally, after generating video frames in VRAM, the GPU would stall, waiting synchronously for the CPU to transfer the data back to host memory. The GPU remained idle while the CPU sequentially executed frame decoding and SSD write operations before a new optimization round could commence.

Our proposed methodology dismantles this bottleneck through three targeted modifications. First, we offloaded post-processing tasks—such as denormalization, associated mathematical operations, and pixel clipping—directly to the GPU, fully utilizing its parallel processing power prior to data transfer (encapsulated within the new **save_results_async** function). Second, we introduced a dedicated background thread pool to handle video encoding and disk I/O. This architectural shift decouples the main script's execution from downstream video processing, enabling true asynchronous operation. Lastly, we converted the video frame data format from 32-bit floating-point (**FP32**) to an 8-bit unsigned integer (**uint8**) representation before initiating the device-to-host transfer. This precise quantization step drastically reduces the data payload traversing the PCIe bus, further minimizing transmission latency.

Optimization Results:

Following the I/O optimizations, empirical results from both Server A and Server B revealed an average inference speedup, reducing the execution time by 15 seconds per case.

6.2.4. *DeepCache Implementation.*

(1)Core Algorithmic Principles of DeepCache

The denoising process in diffusion models unfolds iteratively, wherein spatial features exhibit merely marginal variations between adjacent timesteps. DeepCache capitalizes on this inherent

temporal redundancy to precisely alleviate the massive computational burden of the U-Net architecture. Within the network, the deep layers—tasked with constructing the global structural contours—entail the most intensive and time-consuming matrix operations, yet their feature representations evolve extremely slowly. Conversely, the shallow layers—responsible for generating fine-grained textures—update more rapidly but incur minimal computational overhead.

Consequently, DeepCache abandons the conventional paradigm of executing the complete U-Net forward pass at every timestep. Instead, it bifurcates the denoising trajectory into alternating"Full Step" and "Cached Steps". During a Full Step, the model executes the entire network and caches the computationally expensive deep features. In the subsequent Cached Steps, the algorithm physically bypasses the dense and time-consuming matrix multiplications within the deep layers. It directly retrieves the cached deep representations and exclusively executes the lightweight shallow layers to refresh high-frequency details.

In essence, the profound acceleration achieved by DeepCache stems from its ability to flawlessly substitute highly repetitive and computationally prohibitive deep matrix operations with efficient cache reads. This mechanism successfully offloads the most severe processing burden from the GPU, realizing significant end-to-end speedups.

(2)Analysis of the UnifoLM-WMA Denoising Process and U-Net Architecture

During the video generation denoising process, the model must seamlessly integrate multimodal conditions, spanning images, actions, and text. To underpin this complex joint spatiotemporal denoising task, UnifoLM-WMA employs a highly customized, perfectly symmetric 3D U-Net architecture.

The entire network manifests a classic "U-shaped" topology, wherein the forward propagation of data is delineated into three distinct, functionally specialized stages:

1) **Input Stage (Feature Downsampling and Extraction):** Comprising 13 network tiers, this phase executes progressive downsampling on the massive input volume. It systematically compresses high-resolution surface pixel information into low-resolution, high-dimensional core abstract features.
2) **Bottleneck Stage (Deep Interaction and Fusion):** Situated at the deepest stratum of the U-Net. Here, spatial dimensions are maximally compressed, yet computational density and latency reach their global peak. It shoulders the most computationally intensive spatiotemporal attention matrix operations, orchestrating the deepest semantic interaction across all modalities.
3) **Output Stage (Feature Restoration and Reconstruction):** Symmetrically composed of 13 tiers, this phase progressively upsamples and amplifies the deep abstract representations. By adeptly leveraging Skip Connections, it reintroduces high-frequency details from the Input Stage to ultimately reconstruct high-fidelity, pristine video frames.

(3)DeepCache Optimization Strategy

To overcome the severe PSNR degradation caused by error accumulation inherent in the original static DeepCache mechanism, we deeply refactored the underlying architecture to introduce a Timestep-Aware Dynamic Scheduling System.

This system precisely regulates the operational mechanics of DeepCache through three mutually decoupled control parameters:

1) **Intervention Timing:** This parameter dictates when DeepCache is permitted to activate within the overall denoising trajectory. We enforce full forward passes (Full Steps) during the initial denoising phase to ensure the rigorous construction of the video's underlying physical topology. The caching logic is exclusively initiated during the later fine-tuning stages, thereby maximally preserving the baseline PSNR.

2) **Truncation Depth:** This parameter determines the specific network layer at which the forward pass truncates and returns during a Cached Step. Retaining more shallow layers yields better preservation of mid-level semantic features, whereas an earlier truncation delivers a more aggressive reduction in computational overhead (FLOPs). This provides the system with architectural-level elasticity to navigate the Pareto Optimum between inference acceleration and image fidelity.
3) **Cache Frequency:** This parameter controls how often a complete Full Step is forcefully injected during the DeepCache-enabled phase. By defining a rational interval cycle to serve as an "error calibratio" anchor for deep features, this strategy effectively mitigates the error accumulation stemming from prolonged reliance on outdated caches, striving to ensure the PSNR meets the required evaluation standards.

(4)DeepCache Optimization Results

During empirical evaluations on Server A, we conducted a multi-parameter joint search aimed at maximizing compute utilization. While this approach successfully curtailed certain computational overheads, the PSNR across several test cases still exhibited noticeable degradation. Moving forward, this dynamic system requires further iterative refinement and optimization.

And the optimized results when taking a conservative DeepCache strategy on Server A with an average speedup as 1.217× are summarized in Table 12.

Table 12. *Impact of DeepCache Strategy on Server A Inference, where the baseline performance is detailed in Section 6.3.2. And, the values in the parentheses of the "Test Time" and "Test PSNR" columns respectively represent the speedup ratio relative to the baseline time and the difference to baseline PSNR.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	8m21.168s / 501.168 s (1.474×)	30.1433(-5.498)
	2	8m56.530s / 536.530 s (1.172×)	31.1283(-2.174)
	3	9m3.249s / 543.249 s (1.153×)	30.8455(-8.228)
	4	8m38.673s / 518.673 s (1.215×)	22.4534(-11.675)
unitree_z1_stackbox	1	9m6.876s / 546.876 s (1.250×)	39.7389(-9.175)
	2	9m9.473s / 549.473 s (1.240×)	24.1405(-16.602)
	3	9m14.818s / 554.818 s (1.214×)	38.6058(-11.175)
	4	9m20.097s / 560.097 s (1.214×)	45.2323(-2.028)
unitree_z1_dual_arm_stackbox_v2	1	8m58.534s / 538.534 s (1.180×)	22.5151(-3.553)
	2	8m38.030s / 518.030 s (1.227×)	25.3888(-5.119)
	3	8m45.247s / 525.247 s (1.222×)	23.6633(-8.330)
	4	8m41.169s / 521.169 s (1.224×)	32.2411(-6.988)
unitree_z1_dual_arm_stackbox	1	6m4.289s / 364.289 s (1.195×)	31.7888(-7.343)
	2	6m25.142s / 385.142 s (1.129×)	28.9313(-10.992)
	3	6m20.816s / 380.816 s (1.150×)	34.1748(-2.510)
	4	6m7.895s / 367.895 s (1.186×)	33.3819(-8.129)
unitree_z1_dual_arm_cleanup_pencils	1	6m42.271s / 402.271 s (1.198×)	34.3319(-8.967)
	2	6m30.809s / 390.809 s (1.236×)	39.0612(-9.618)
	3	6m29.452s / 389.452 s (1.237×)	30.3996(-11.759)
	4	6m30.723s / 390.723 s (1.227×)	37.1565(-9.249)
Average	-	7m54.263s / 474.263 s (1.217×)	31.7661

(5)Analysis of PSNR loss

The fidelity with which model-generated videos represent the physical world is fundamentally governed by the underlying algorithmic architecture and the training corpus. However, it is also susceptible to variations arising from different hardware architectures and software library invocations. Consequently, evaluating the identical baseline across disparate platforms inherently yields discrepancies in PSNR values. Using our local baseline results as the benchmark, the observed degradation in PSNR upon introducing the DeepCache strategy can be analyzed as follows:

The DeepCache strategy exploits the temporal coherence of the U-Net architecture, specifically the minimal variance in intermediate activations within its deeper layers during the denoising process . Furthermore, it leverages the behavior of the UnifoLM-WMA model in the later stages of denoising, where the focus shifts toward refining high-frequency details across video frames, resulting in highly stable neural network feature maps. By caching and reusing the intermediate activations from a preceding denoising step to substitute those that would otherwise be newly computed, this approach significantly curtails the computational overhead.

Fundamentally, DeepCache achieves acceleration by substituting exact computations with approximated cached data. This inherent nature of approximation inevitably introduces a degree of fidelity loss during video generation.

In practice, however, we can qualitatively assess the variance dynamics of the deep intermediate activations based on the model's specific denoising trajectory. This enables the design of an adaptive and well-grounded DeepCache scheduling strategy to minimize the approximation-induced degradation. The solution we proposed represents a relatively straightforward implementation of this concept. Nevertheless, during the current tuning phase, we could not guarantee that the PSNR threshold is met across all test cases. Future work will necessitate extensive empirical testing across multiple parameter combinations to identify the optimal conservative parameter configuration, or to develop a more sophisticated scheduling policy.

Conversely, within the Server B evaluation environment, we deployed a traditional DeepCache configuration (i.e., executing a complete forward pass alternately every two denoising steps). **We synergistically fused this caching mechanism with foundational low-level optimizations, including single-node NUMA affinity pinning, mixed-precision computation, and I/O enhancements. The resulting experimental data is presented in Table 13 with an average per-case speedup of 3.596×.**

Intriguingly, in the unoptimized baseline evaluations, we observed that the robot in the generated videos—when compared to the ground-truth reference—exhibits temporal looping (repeating the same action) and a delayed execution of subsequent steps, yet it still maintains a notably high PSNR. Counterintuitively, upon integrating the DeepCache strategy, certain test cases on Server B demonstrated an anomalous increase in PSNR. This paradox likely occurs because DeepCache, by reusing historical feature representations, inadvertently acts as a regularizer that mitigates the issue of parameter jitter during the generation process.

6.2.5. *FlashAttention Recomputation.*

In processing high-resolution, multi-frame video features, the standard self-attention mechanism encounters a severe data-transfer bottleneck. Modern GPU architectures utilize a hierarchical memory system comprising high-capacity but slow global memory (HBM) and ultra-fast but severely limited on-chip cache (SRAM). During execution, standard attention operators require compute cores to fetch inputs from HBM to calculate the Query (Q), Key (K), and Value (V) matrices within the SRAM. Due to strict SRAM capacity limits, the hardware cannot retain the subsequent attention score matrix, which explodes quadratically ($\mathcal{O}(N^2)$) with the sequence length N. Consequently, the system is forced to materialize and write this massive intermediate tensor back to the slow

Table 13. *Impact of DeepCache on Server B Inference, where the baseline performance is detailed in Section 6.3.2.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	6m26.336s / 386.336 s (3.714×)	18.792(0.270)
	2	6m28.452s / 388.452 s (3.708×)	19.951(1.408)
	3	6m24.274s / 384.274 s (3.747×)	20.086(0.129)
	4	6m24.506s / 384.506 s (3.737×)	18.302(0.528)
unitree_z1_stackbox	1	6m47.822s / 407.822 s (3.809×)	21.918(-2.065)
	2	6m49.017s / 409.017 s (3.807×)	21.550(1.493)
	3	6m46.275s / 406.275 s (3.826×)	22.371(0.665)
	4	6m46.510s / 406.510 s (3.831×)	22.511(-2.767)
unitree_z1_dual_arm_stackbox_v2	1	6m36.929s / 396.929 s (3.649×)	19.862(-0.156)
	2	6m33.524s / 393.524 s (3.680×)	19.816(0.924)
	3	6m34.833s / 394.833 s (3.672×)	20.627(1.800)
	4	6m35.834s / 395.834 s (3.658×)	19.443(0.100)
unitree_z1_dual_arm_stackbox	1	4m52.944s / 292.944 s (3.285×)	22.674(-0.143)
	2	4m53.256s / 293.256 s (3.293×)	25.100(0.521)
	3	4m51.872s / 291.872 s (3.295×)	23.510(-2.757)
	4	4m52.891s / 292.891 s (3.291×)	25.153(-0.242)
unitree_z1_dual_arm_cleanup_pencils	1	5m8.495s / 308.495 s (3.478×)	21.186(-0.046)
	2	5m9.548s / 309.548 s (3.479×)	19.644(-2.897)
	3	5m8.966s / 308.966 s (3.472×)	22.592(2.596)
	4	5m9.415s / 309.415 s (3.480×)	21.113(-1.752)
Average	-	5m58.085s / 358.085 s (3.596×)	21.310(-0.1197)

HBM. In the ensuing Softmax normalization and V-matrix weighting steps, compute units must repeatedly cycle through reading, processing, and writing back this colossal matrix. This exhaustive data shuttling leaves high-speed Tensor Cores idling while waiting for memory transfers, severely degrading overall inference speed.

To eradicate this memory-transfer latency, we implemented the FlashAttention acceleration mechanism. Its core paradigm leverages operator fusion and tiling strategies to fundamentally bypass the instantiation and storage of the massive intermediate matrix. By partitioning the Q, K, and V matrices into granular blocks perfectly sized for SRAM, FlashAttention executes the dot-product, Softmax scaling, and weighted accumulation entirely on-chip in a single, uninterrupted pass. This optimized computational flow never outputs $\mathcal{O}(N^2)$ intermediate tensors to global memory, writing back only the final results. Such architectural refactoring effectively truncates redundant data round-trips, successfully shifting the system's operational bottleneck from "memory-bandwidth bound" to "pure compute bound," thereby achieving drastic inference speedups with zero degradation in generative precision.

To address the ubiquitous spatiotemporal attention bottlenecks within the model, our team executed an invasive, code-level refactoring across core backbone components, including ae_modules.py, attention.py, condition.py, conditional_unet1d.py, and resampler.py. The fundamental principle of this architectural overhaul was to strip away inefficient Python-level mathematical computations. Through rigorous alignment of tensor manifolds, we comprehensively redirected the core computational workloads to the underlying FlashAttention hardware-accelerated kernels.

For Server A, we conducted an isolated optimization evaluation, applying this specific strategy exclusively while disabling all other enhancements. Under this controlled setup,

Table 14. *Impact of FlashAttention Recomputation on Server A Inference, where the baseline performance is detailed in Section 6.3.2.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	8m53.628s / 533.628 s (1.384×)	36.567(0.926)
	2	8m53.426s / 533.426 s (1.178×)	38.455(5.152)
	3	8m51.673s / 531.673 s (1.178×)	36.643(-2.431)
	4	8m52.845s / 532.845 s (1.182×)	33.950(-0.178)
unitree_z1_stackbox	1	9m30.271s / 570.271 s (1.199×)	48.641(-0.273)
	2	9m38.091s / 578.091 s (1.179×)	44.681(3.938)
	3	9m37.282s / 577.282 s (1.167×)	49.681(-0.100)
	4	9m33.556s / 573.556 s (1.185×)	47.173(-0.088)
unitree_z1_dual_arm_stackbox_v2	1	8m59.184s / 539.184 s (1.179×)	26.358(0.290)
	2	8m59.727s / 539.727 s (1.178×)	26.612(-3.896)
	3	9m8.605s / 548.605 s (1.170×)	32.746(0.752)
	4	9m6.011s / 546.011 s (1.168×)	39.854(0.625)
unitree_z1_dual_arm_stackbox	1	6m21.789s / 381.789 s (1.140×)	45.437(6.306)
	2	6m18.484s / 378.484 s (1.149×)	39.210(-0.712)
	3	6m17.614s / 377.614 s (1.160×)	36.596(-0.089)
	4	6m17.295s / 377.295 s (1.156×)	40.913(-0.597)
unitree_z1_dual_arm_cleanup_pencils	1	7m1.541s / 421.541 s (1.144×)	45.610(2.311)
	2	6m50.965s / 410.965 s (1.176×)	47.229(-1.450)
	3	6m52.423s / 412.423 s (1.168×)	44.718(2.560)
	4	7m2.009s / 422.009 s (1.136×)	46.357(-0.048)
Average	-	8m9.321s / 489.321 s (1.179×)	40.372(0.449)

we achieved an average speedup of 1.179× across all test cases. The detailed performance metrics are summarized in Table 14.

For Server B, after cumulatively applying single-node NUMA pinning, mixed-precision optimization, I/O enhancements, FlashAttention recomputation, and the DeepCache strategy, the experimental results are presented in Table 15.

FlashAttention recomputation is hardware-aware exact optimization. It drastically improves memory and bandwidth efficiency while guaranteeing the strict mathematical equivalence of the forward pass, thereby leaving objective evaluation metrics (PSNR) almost unaffected.

6.3. **Experiments.**

6.3.1. *Environment Setup.*

(1) Server A

- **Hardware Configuration:** Unless otherwise stated, the system is powered by an AMD EPYC 7542 processor (configured with 16 logical cores), 64GB of RAM, and a single NVIDIA GeForce RTX 4090 GPU (24GB VRAM) based on the Ada Lovelace architecture.
- **Software Environment:** The software stack comprises Ubuntu 22.04 LTS, with a system-level CUDA 13.0 and an environment-specific nv-CUDA 12.1. The computational framework is built on Python 3.10.18, PyTorch 2.3.1, and cuDNN 8902.

(2) Server B

Server B served as the initial platform for baseline benchmarking and validation of the Embodied World Model Optimization. Although the baseline test results on this machine did not fully meet

Table 15. *Impact of FlashAttention on Server B Inference, where the baseline performance is detailed in Section 6.3.2.*

Scenario	Case	Test Time	Test PSNR
unitree_g1_pack_camera	1	6m2.708s / 362.708 s (3.956×)	18.195(-0.328)
	2	6m4.189s / 364.189 s (3.955×)	19.724(1.181)
	3	6m2.248s / 362.248 s (3.975×)	20.801(0.843)
	4	6m2.315s / 362.315 s (3.966×)	18.409(0.634)
unitree_z1_stackbox	1	6m24.343s / 384.343 s (4.041×)	24.425(0.443)
	2	6m26.186s / 386.186 s (4.032×)	20.599(0.542)
	3	6m21.553s / 381.553 s (4.074×)	21.582(-0.124)
	4	6m22.799s / 382.799 s (4.068×)	22.560(-2.717)
unitree_z1_dual_arm_stackbox_v2	1	6m14.874s / 374.874 s (3.863×)	20.311(0.293)
	2	6m11.942s / 371.942 s (3.893×)	19.773(0.882)
	3	6m14.471s / 374.471 s (3.871×)	20.050(1.223)
	4	6m14.022s / 374.022 s (3.872×)	19.706(0.363)
unitree_z1_dual_arm_stackbox	1	4m34.408s / 274.408 s (3.507×)	21.951(-0.866)
	2	4m36.214s / 276.214 s (3.496×)	25.115(0.536)
	3	4m35.233s / 275.233 s (3.495×)	23.550(-2.717)
	4	4m33.352s / 273.352 s (3.527×)	25.263(-0.132)
unitree_z1_dual_arm_cleanup_pencils	1	4m49.005s / 289.005 s (3.713×)	21.718(0.486)
	2	4m49.090s / 289.090 s (3.725×)	20.385(-2.156)
	3	4m49.847s / 289.847 s (3.701×)	22.593(2.597)
	4	4m50.073s / 290.073 s (3.712×)	21.016(-1.849)
Average	-	5m36.944s / 336.944 s (3.822×)	21.386(-0.0437)

the official competition requirements, the experimental data provided critical insights for auxiliary analysis and cross-platform performance profiling.

- **Hardware Configuration:** This server features a mature dual-socket NUMA architecture, equipped with two Intel Xeon E5-2680 v4 processors, providing a total of 28 physical cores and 56 logical threads. It is configured with 128GB of DDR4 RAM (approx. 125GiB usable). For accelerated computing, the system deploys dual NVIDIA GeForce RTX 3090 GPUs, each offering 24GB of GDDR6X VRAM.
- **Software Environment:** The software stack comprises Ubuntu 22.04 LTS, with a system-level CUDA 12.2 and an environment-specific nv-CUDA 12.1. The computational framework is built on Python 3.10.18, PyTorch 2.3.1, and cuDNN 8902.

(3) Comparison Methods

- **Baseline Testing:** The environment was configured in strict accordance with the competition guidelines provided by the Organizing Committee, supplemented by the official README documentation from the Unitree GitHub repository. Baseline experimental results were subsequently generated under these standardized conditions.
- **Optimization Implementation:** The primary objective of our optimization was to minimize model inference latency to the greatest extent possible, while strictly adhering to the competition's quality mandate: ensuring that the Peak Signal-to-Noise Ratio (PSNR) for every test case remains no less than the 25 dB threshold.

6.3.2. *Evaluation Results.*

(1) Server A

The Baseline results on Server A are summarized in Table 16, and the Optimized Results on Server A are presented in Table 17. **Following the deployment of our optimization strategies(an integrated optimization strategy leveraging mixed precision, I/O enhancements, and FlashAttention recomputation), the system achieved a mean speedup of $1.407\times$ across all test cases.**

The analysis of how this integrated scheme accelerates model inference is detailed in Section 6.2. Due to the inherent insensitivity of deep neural networks to lower-order fractional bits in floating-point operations, the adoption of mixed-precision optimization yields minimal PSNR degradation; in fact, it results in an average PSNR increase of approximately 0.9 dB per case. Similarly, FlashAttention Recomputation performs a strictly mathematically equivalent transformation, exerting a negligible impact on PSNR. Furthermore, I/O enhancements optimize data transmission pathways without altering the underlying mathematical computational graph. **Consequently, the proposed framework successfully preserves video fidelity: the baseline average PSNR stands at 39.722 dB, whereas the optimized average reaches a slightly elevated 41.892 dB, demonstrating an overall improvement of nearly 2 dB.**

Table 16. *Baseline Performance on Server A*

Scenario	Case	Baseline Time	Baseline PSNR
unitree_g1_pack_camera	1	12m18.586s / 738.586 s	35.641
	2	10m28.571s / 628.571 s	33.303
	3	10m26.364s / 626.364 s	39.074
	4	10m30.067s / 630.067 s	34.128
unitree_z1_stackbox	1	11m23.748s / 683.748 s	48.914
	2	11m21.399s / 681.399 s	40.743
	3	11m13.402s / 673.402 s	49.781
	4	11m19.914s / 679.914 s	47.261
unitree_z1_dual_arm_stackbox_v2	1	10m35.479s / 635.479 s	26.068
	2	10m35.605s / 635.605 s	30.508
	3	10m41.678s / 641.678 s	31.994
	4	10m37.786s / 637.786 s	39.229
unitree_z1_dual_arm_stackbox	1	7m15.180s / 435.180 s	39.131
	2	7m14.908s / 434.908 s	39.923
	3	7m17.951s / 437.951 s	36.685
	4	7m16.239s / 436.239 s	41.511
unitree_z1_dual_arm_cleanup_pencils	1	8m2.062s / 482.062 s	43.299
	2	8m3.155s / 483.155 s	48.679
	3	8m1.702s / 481.702 s	42.159
	4	7m59.364s / 479.364 s	46.405
Average	-	9m38.158s / 578.158 s	39.722

(2) Server B

Since the baseline results on this server do not satisfy the Organizing Committee's mandated threshold ($PSNR < 25dB$), the following data is provided for analytical reference only. **These results were obtained by integrating optimizations such as single-node NUMA pinning, mixed-precision arithmetic, I/O enhancements, DeepCache implement and FlashAttention.**

The baseline results on Server B are presented in Table 18, and the optimization performance on Server B are summarized in Table 19.

Table 17. *Optimization Performance on Server A*

Scenario	Case	Test PSNR	Test Time	Velocity
unitree_g1_pack_camera	1	42.803(7.162)	7m25.468s / 445.468 s	1.658×
	2	38.678(5.375)	7m29.521s / 449.521 s	1.398×
	3	34.311(-4.762)	7m27.581s / 447.581 s	1.399×
	4	36.162(2.034)	7m26.605s / 446.605 s	1.411×
unitree_z1_stackbox	1	46.963(-1.951)	7m56.135s / 476.135 s	1.436×
	2	47.972(7.229)	7m58.630s / 478.630 s	1.424×
	3	49.644(-0.137)	8m0.076s / 480.076 s	1.403×
	4	46.982(-0.279)	7m55.432s / 475.432 s	1.430×
unitree_z1_dual_arm_stackbox_v2	1	27.133(1.066)	7m36.540s / 456.540 s	1.392×
	2	29.992(-0.516)	7m33.582s / 453.234 s	1.402×
	3	30.558(-1.436)	7m39.803s / 459.803 s	1.396×
	4	43.003(3.774)	7m36.808s / 456.808 s	1.396×
unitree_z1_dual_arm_stackbox	1	45.104(5.973)	5m19.619s / 319.619 s	1.362×
	2	42.239(2.316)	5m18.587s / 318.587 s	1.365×
	3	47.758(11.073)	5m18.785s / 318.785 s	1.374×
	4	47.505(5.995)	5m20.033s / 320.033 s	1.363×
unitree_z1_dual_arm_cleanup_pencils	1	43.359(0.060)	5m46.849s / 346.849 s	1.390×
	2	47.533(-1.146)	5m49.216s / 349.216 s	1.384×
	3	45.321(3.163)	5m49.795s / 349.795 s	1.377×
	4	44.821(-1.584)	5m47.695s / 347.695 s	1.379×
Average	-	41.892(1.969)	6m49.820s / 409.820s	1.407×

Initial baseline evaluations on Server B revealed that the majority of test cases failed to meet the required PSNR threshold of 25 dB, with an average of approximately 21 dB. Suspecting environment misconfiguration or incorrect parameter settings, we conducted extensive debugging: verifying input files (.h5 and initial images), reinstalling dependencies and drivers, adjusting CUDA environment variables, and cross-checking inference script parameters against the official documentation. Despite these exhaustive efforts, the PSNR values remained unchanged, effectively ruling out software-related issues.

To isolate the root cause, we migrated the identical baseline test to alternative hardware platforms, including a server equipped with an RTX 4090 (Ada Lovelace architecture) and another with an A800 (enhanced Ampere architecture). On both machines, all test cases consistently met the competition's PSNR requirement. This strongly suggested that the issue was specific to the GPUs used in Server B.

Server B is configured with dual RTX 3090s (Ampere architecture), whereas the RTX 4090 and A800 represent newer or refined architectures. We hypothesize that discrepancies in floating-point execution details across GPU generations—such as instruction scheduling, rounding modes, or Tensor Core numerical behavior—may account for the observed variance. While such differences are negligible for general-purpose workloads, in large-scale neural network inference they can accumulate across layers, leading to pixel-level deviations

that manifest as measurable PSNR degradation. Although the RTX 3090 remains a capable GPU, its computation patterns for numerically sensitive operators may diverge from those of newer architectures, resulting in suboptimal baseline performance.

Notably, despite Server B's baseline PSNR falling short of the competition standard, subsequent optimization experiments showed that PSNR remained stable at approximately 21.4 dB, closely matching the baseline. This indicates that the optimization techniques introduced no additional quality degradation. Consequently, the speedup measurements obtained on this platform retain meaningful comparative value, offering reliable insights into the relative efficacy of different optimization strategies.

Table 18. *Baseline Performance on Server B*

Scenario	Case	Baseline Time	Baseline PSNR
unitree_g1_pack_camera	1	23m55.042s / 1435.042 s	18.5229
	2	24m0.467s / 1440.467 s	18.5432
	3	24m0.049s / 1440.049 s	19.9578
	4	23m56.881s / 1436.881 s	17.7746
unitree_z1_stackbox	1	25m53.235s / 1553.235 s	23.9826
	2	25m57.084s / 1557.084 s	20.0572
	3	25m54.368s / 1554.368 s	21.7059
	4	25m57.163s / 1557.163 s	25.2775
unitree_z1_dual_arm_stackbox_v2	1	24m8.266s / 1448.266 s	20.0181
	2	24m8.073s / 1448.073 s	18.8915
	3	24m9.671s / 1449.671 s	18.8271
	4	24m8.079s / 1448.079 s	19.3431
unitree_z1_dual_arm_stackbox	1	16m2.436s / 962.436 s	22.8169
	2	16m5.721s / 965.721 s	24.5789
	3	16m1.834s / 961.834 s	26.2675
	4	16m4.021s / 964.021 s	25.3952
unitree_z1_dual_arm_cleanup_pencils	1	17m53.061s / 1073.061 s	21.2321
	2	17m56.892s / 1076.892 s	22.5411
	3	17m52.589s / 1072.589 s	19.9960
	4	17m56.808s / 1076.808 s	22.8648
Average	-	21m36.087s / 1296.087 s	21.4297

6.4. **Optimization Plans.**

6.4.1. *Operator Fusion.*

In the future, we will explore operator fusion as a potential implementation to optimize model inference speed. By refactoring the computation graph, multiple logically independent discrete operators will be merged into a single hardware execution kernel, thereby breaking the current "compute-store-reload" serial execution bottleneck. This integrates fragmented calculation tasks into a streamlined execution path, enhancing low-level execution continuity by reducing kernel launch frequency and CPU-GPU scheduling overhead.

The inference optimization achieved by operator fusion stems from two primary factors:

- **Reduction of Scheduling Overhead:** It minimizes kernel launch frequency and the associated CPU-GPU scheduling latency.

Table 19. *Optimization Performance on Server B*

Scenario	Case	Test PSNR	Test Time	Velocity
unitree_g1_pack_camera	1	18.195(-0.328)	6m2.708s / 362.708 s	3.956×
	2	19.724(1.181)	6m4.189s / 364.189 s	3.955×
	3	20.801(0.843)	6m2.248s / 362.248 s	3.975×
	4	18.409(0.634)	6m2.315s / 362.315 s	3.966×
unitree_z1_stackbox	1	24.425(0.443)	6m24.343s / 384.343 s	4.041×
	2	20.599(0.542)	6m26.186s / 386.186 s	4.032×
	3	21.582(-0.124)	6m21.553s / 381.553 s	4.074×
	4	22.560(-2.717)	6m22.799s / 382.799 s	4.068×
unitree_z1_dual_arm_stackbox_v2	1	20.311(0.293)	6m14.874s / 374.874 s	3.863×
	2	19.773(0.882)	6m11.942s / 371.942 s	3.893×
	3	20.050(1.223)	6m14.471s / 374.471 s	3.871×
	4	19.706(0.363)	6m14.022s / 374.022 s	3.872×
unitree_z1_dual_arm_stackbox	1	21.951(-0.866)	4m34.408s / 274.408 s	3.507×
	2	25.115(0.536)	4m36.214s / 276.214 s	3.496×
	3	23.550(-2.717)	4m35.233s / 275.233 s	3.495×
	4	25.263(-0.132)	4m33.352s / 273.352 s	3.527×
unitree_z1_dual_arm_cleanup_pencils	1	21.718(0.486)	4m49.005s / 289.005 s	3.713×
	2	20.385(-2.156)	4m49.090s / 289.090 s	3.725×
	3	22.593(2.597)	4m49.847s / 289.847 s	3.701×
	4	21.016(-1.849)	4m50.073s / 290.073 s	3.712×
Average	-	21.386(0.0437)	5m36.944s / 336.944 s	3.822×

- **Enhanced Parallelism and Hardware Utilization:** The resulting long instruction streams allow the compiler to perform deeper Instruction-Level Parallelism (ILP) optimizations. This reduces hardware scheduling costs for Streaming Multiprocessors (SMs) when switching between multiple small tasks, enabling compute units such as Tensor Cores to maintain a sustained high-throughput state.

From the perspective of flame graph analysis, the left side exhibits a broad horizontal cascade (layers of **libcuda.so**) while the right side shows dense, fine-grained vertical stacks (scheduling traces of **c10::** and **at::**), reflecting significant operator fragmentation and kernel launch overhead. This indicates that the system is in a typical CPU-bound state: numerous small, discrete memory-bound operations frequently interrupt the GPU execution pipeline, causing the CPU to spend more time on low-level driver API calls and context switches than the actual computation on CUDA cores.

Implementing fine-grained operator fusion can reconstruct the spatio-temporal locality of data flow. By explicitly rewriting frequently occurring composite operator blocks (such as normalization and activation functions) into a single kernel, repeated read-write operations of intermediate activations to GPU HBM can be eliminated. This keeps data circulation closed within low-latency SRAM, thereby accelerating model inference optimization.

6.4.2. *Multi-GPUs.*

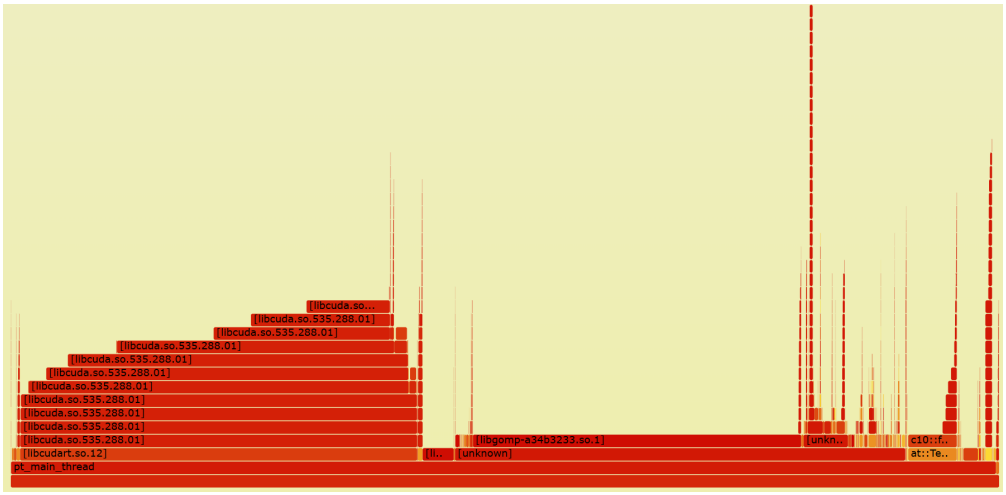


Figure 9. *Inference process flame graph*

In the practical implementation of model inference, the process consists of n iterations, as specified by the n_iter parameter in the run_world_model_interaction.sh script. Each iteration can be further divided into two sequential stages: first, the VLA model generates an action sequence; second, the world model takes this action sequence as input and produces corresponding frames. The output of each iteration serves as the input for the next, making the inference process of unifolm-wma inherently sequential.

However, with respect to the attention computations involved in different parts of the denoising process, an optimization can be envisioned for a dual-GPU setup. Specifically, the initialization data required for model inference is simultaneously loaded into both GPUs. One implementation approach involves the CPU first reading the data from an SSD into memory, then transferring it to the GPU local to the CPU, after which this GPU copies the data to the other GPU. Subsequently, both GPUs perform denoising in parallel, each handling their respective portion of the attention computation. The computed attention results are then distributed across GPUs via blocking operations to facilitate subsequent steps, thereby enabling parallel processing of the attention computation workload.

6.5. **Conclusion.**

In the optimization of UnifoLM-WMA inference, our team systematically evaluated several strategies, including single-node NUMA affinity, mixed-precision arithmetic, I/O enhancements, FlashAttention, and DeepCache. To maximize inference throughput while strictly adhering to the mandated fidelity threshold ($PSNR > 25dB$), we adopted a balanced trade-off approach. Our final deployment integrates mixed-precision, I/O optimizations, and FlashAttention; however, the DeepCache strategy was excluded to preserve baseline quality, and operator fusion remains in the exploratory phase. Under this configuration, the system achieved a mean acceleration ratio of $1.426\times$. Moving forward, we intend to further refine and synergistically combine diverse optimization methodologies tailored to the specific algorithmic architecture, aiming to enhance inference efficiency while ensuring full PSNR compliance.

7. Optimization for Numerical Relativity Code AMSS-NCKU

7.1. **Introduction.**

General Relativity is the fundamental theory describing gravity. Although it has been established for over a century, human understanding of it remains limited by the extreme complexity of the Einstein field equations, which constitute one of the most challenging systems of partial differential equations in physics. To analyze gravitational wave detection data, obtaining accurate numerical solutions describing Binary Black Hole (BBH) mergers is crucial. Gravitational wave detection was first realized in 2015 and awarded the Nobel Prize in Physics in 2017. This breakthrough was largely attributed to the numerical solutions of the Einstein field equations. Software used to numerically solve these equations is collectively referred to as "numerical relativity codes". AMSS-NCKU is one of the few publicly available codes of this kind worldwide [4]. Its core technical features include:

- **Numerical Methods:** Adopting finite-difference methods to solve the equations.
- **Multi-scale Handling:** Utilizing Adaptive Mesh Refinement (AMR) technology to handle the inherent multi-scale characteristics of General Relativity.
- **Parallel Strategy:** Adopting a Domain Decomposition strategy combined with MPI for parallel processing.

In this competition, our main task is to perform numerical simulations of binary black hole mergers and use technical means to minimize the total runtime of the simulation. It should be noted that we integrated multiple software components (such as `ccache`) into a Docker container. These components cache the compiled content, thereby shortening the time required for the compilation phase in subsequent runs, even with partial modifications.

By analyzing the workflow, reviewing the work logs, and conducting flame graph analysis on the time-consuming steps of the project, we determined the direction for subsequent optimizations. Through methods such as adjusting compiler parameters, changing compilation options, inlining and vectorizing hotspot functions, grid partitioning, memory alignment, and asynchronous communication, **we reduced the total wall-clock time from 33.21 hours to 21.157 hours, achieving a speedup of 1.539** $\times$.

Meanwhile, through further exploration and analysis, we discovered potential optimization directions such as incomplete memory alignment optimization, dynamic load balancing, and the mixed use of MPI and OpenMP. These will be the focus of our future work, with the aim of further shortening the runtime and achieving better optimization results.

7.2. **Optimization Methods.**

To illustrate the impact of our modifications within the overall system, the specific locations of these enhancements are annotated in the "Workflow Optimization Map". The designated markers in the diagram represent the following technical optimizations:

- ① Compiler instruction set optimization;
- ② Inter-procedural (cross-file) inlining;
- ③ Loop tiling (blocking) optimization;
- ④ Computation-communication overlap and non-blocking communication;
- ⑤ Memory alignment;
- ⑥ Utilization of compiler caching tools;
- ⑦ Selection of an optimal process count tailored to the target hardware platform during the input phase.

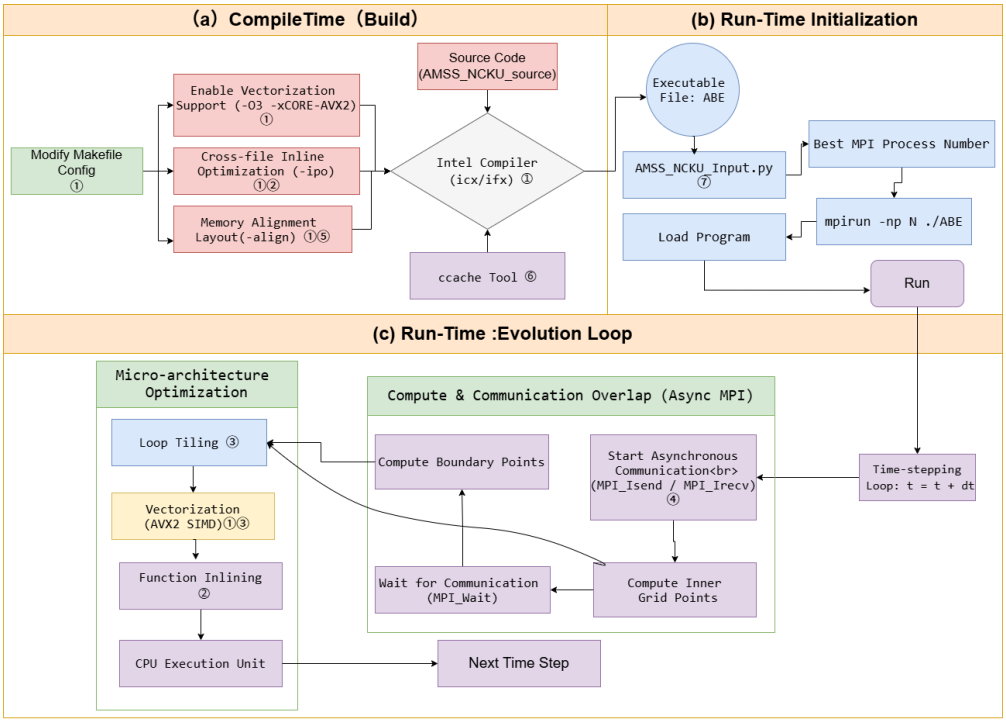


Figure 10. *Workflow Optimization Map*

Prior to discussing the specific optimization strategies, it is essential to establish the characteristics of our baseline performance. During the initial successful execution, we observed that the per-step evolution time remained consistently at approximately 51 seconds for the first 80 percent of the total runtime. As the simulation progressed to the 1971-th step, the grid boundaries of the two black holes began to merge. This confluence led to the gradual elimination of redundant grid points and a sharp reduction in MPI communication overhead, causing the per-step time to decrease and eventually stabilize at roughly 40 seconds. To ensure a rigorous and consistent evaluation of subsequent optimizations, we have adopted the average per-step time within the first 5.0M evolution time units as the primary metric. Furthermore, the relevant data for the baseline performance is summarized in Table 20.

Table 20. *Baseline Performance*

Methods	**Final Evolution Time**	**Cores**	**Total Project Time**	**Evolve Time**	**Single Step / Avg**
Baseline	1000.0M(2400 steps)	28	119556s	118797s	From 40.1s to 53.6s (Avg: 49.5s)

7.2.1. *Compiler Parameter Adjustment.*

To establish a solid foundation for performance enhancements, we systematically overhauled the compilation environment and fine-tuned compiler heuristics to maximize hardware utilization:

(1) **Compiler Toolchain Migration:** We transitioned the primary build system from the GNU Compiler Collection (`gcc/gfortran`) to the Intel oneAPI toolchain (`icx/mpiifx`). Intel compilers are inherently tailored to generate highly optimized machine code for Xeon processors, demonstrating superior binary compatibility and intrinsic library performance, particularly within mixed C++/Fortran codebases.

(2) **Hardware-Level Vectorization (`-xCORE-AVX2`):** We explicitly instructed the compiler to target the AVX2 instruction set. During the computationally intensive hyperbolic evolution phase of the Einstein field equations, this allows the CPU to process four double-precision floating-point numbers simultaneously within a single clock cycle using 256-bit registers. Given that the target Intel Xeon E5-2680 v4 (Broadwell) architecture does not support the wider AVX-512 instructions, fully exploiting AVX2 is critical, yielding a theoretical peak performance several times higher than strictly scalar execution.

After implementing initial modifications to the compilation configuration, we achieve a 1.390× speed-up as shown in Table 21.

Table 21. *Compiler parameter adjustment optimization performance*

Final Evolution Time	Cores	Total Project Time	Total Evolve Time	Single Step Avg
5.0M	16	1044.48s	462.841s	35.603s

7.2.2. *Inline Function.*

The optimization of inline functions and computational kernels focuses on stripping away memory redundancy, function call overhead, and unnecessary diagnostic logic to streamline the critical path:

(1) **Elimination of Ghost Copying:** We removed the temporary array `fh` and the `call symmetry_bd(...)` invocation in `diff_new.f90`. This eliminates an $O(N^3)$ memory copy overhead. The kernel now directly accesses the original array `f`, handling boundary conditions via inlined logic.

(2) **Fast/Slow Path Strategy:** To maintain performance without sacrificing safety, we split the computation into a **Fast Path** for the core region (pure calculation without branches) and a **Slow Path** for boundaries (with symmetry checks).

(3) **Loop Fusion:** We aggressively fused the derivative calculations for the X, Y, and Z directions into a single, unified `i` loop. This structural transformation achieves a "load once, compute multiple times" paradigm. By consolidating these operations, we drastically reduced redundant memory fetches, thereby maximizing both L1 Cache utilization and hardware register reuse rates.

(4) **Sanity Check Overhead Reduction:** We identified a massive performance bottleneck at the entry of the RHS evaluation routines (e.g., `bssn_rhs.f90`, `bssn_rhs_ss.f90`, `bssnE_Scalar_rhs.f90`). The legacy code executed a global `sum(...)` operation across more than twenty 3D arrays for `NaN` sanity checking on every single call to `compute_rhs_bssn`. We wrapped this extremely expensive validation step within a preprocessor macro (`#ifdef RHS_SANITY_CHECK`), disabling it by default for production runs. This pragmatic adjustment eliminated a severe memory-scan hotspot without altering the mathematical results.

7.2.3. *Grid Partition Optimization.*

To address the memory-bound nature of the 3D stencil computation, we implemented a "Grid Partition" strategy using "Cache Blocking":

(1) **Tiling Implementation:** In `diff_new.f90` and `lopsidediff.f90`, we transformed the loop structure by adding outer loops kk and jj. The domain is partitioned into tiles of size $32 \times 32 \times N_x$.

(2) **Locality Improvement:**

$$\text{Loop Structure: } \texttt{do kk} \rightarrow \texttt{do jj} \rightarrow \texttt{do k} \rightarrow \texttt{do j} \rightarrow \texttt{do i} \tag{7.1}$$

This structure ensures that the data planes required for the Z-direction stencil ($k \pm 1$) remain resident in the L2 cache, significantly reducing the cache miss rate caused by large strided accesses.

(3) **Empirical Validation (Cache Profiling):** To empirically validate the effectiveness of our spatial locality optimizations, we conducted a direct cache performance comparison using `perf` hardware counters over a 20-second sampling window. The comparative results demonstrate a profound improvement in memory subsystem utilization:

- **L1 Data Cache (L1 D-Cache):** The L1 cache miss rate plummeted from 10.83% to 5.54%. Total L1 load misses were reduced by approximately 52.5% (from 32 billion to 15.2 billion accesses). This confirms the success of the aforementioned loop fusion and register reuse strategies.
- **Last Level Cache (LLC / L3):** The impact of cache blocking (tiling) is most evident here. Total LLC memory requests dropped significantly by 56% (from 5.16 billion to 2.24 billion), and LLC load misses decreased by 59% (from 21.6 billion to 884 million). This indicates that the 32×32 working set successfully resides in the L2/L3 caches, drastically alleviating the main memory (DRAM) bandwidth bottleneck.
- **Instruction Throughput (IPC):** As a direct consequence of reduced memory stall cycles, the overall Instructions Per Cycle (IPC) increased by 18.3% (from 0.82 to 0.97), yielding a much faster execution despite a slight increase in total instruction count caused by the tiling loop overhead.

Table 22 presents the data obtained after implementing Inlining, Vectorization, and Grid Tiling optimizations, building upon the previously established configurations.

Table 22. *Performance of implementing optimization methods*

Final Evolution Time	Cores	Total Project Time	Total Evolve Time	Single Step Avg
5.0M	16	980.5s	451.0s	34.696s

7.2.4. *Memory Alignment.*

To fully exploit the vectorization capabilities of the Broadwell CPU architecture, we enforced strict 64-byte data alignment by introducing the `_mm_malloc` allocator within `Block.C`, tightly coupled with the `-align array64byte` compiler flag. This strategy is fundamentally driven by two hardware-level characteristics:

(1) **Instruction Set Architecture (ISA) Efficiency:** At the instruction level, AVX2 SIMD operations exhibit inherently lower latency when accessing aligned memory addresses. Strict alignment eliminates the additional micro-operation (micro-op) cycles required by the hardware to execute unaligned load and store instructions, thereby streamlining the execution pipeline.

(2) **Memory Hierarchy & Cache Utilization:** Given that the underlying CPU cache line size is exactly 64 bytes, aligning the data boundaries ensures that contiguous grid point arrays do not straddle across two separate cache lines (preventing cache line splits). This precise

alignment drastically reduces the number of required memory bus transactions and maximizes the effective data fill density across the cache hierarchy(particularly within the L3 cache), resulting in a significant throughput enhancement for the entire memory subsystem.

Table 23. shows the results after applying memory optimizations on top of the several preceding optimization methods.

Table 23. *Performance of Implementing Memory Alignment*

Final Evolution Time	Cores	Total Project Time	Total Evolve Time	Single Step Avg
5.0M	16	961.961s	441.15s	33.935s

7.2.5. *Non-blocking & Request Separation.*

To alleviate MPI communication bottlenecks and hide network latency, we overhauled the synchronization routines in `Parallel.C` (`transfer` and `transfermix`) using advanced asynchronous communication strategies and buffer caching:

(1) **Non-blocking Paradigm & Queue Separation:** We transitioned from blocking communications to a fully non-blocking paradigm. The request queues were explicitly split into independent `recv_reqs` and `send_reqs`. By systematically posting all `MPI_Irecv` calls prior to data packing and `MPI_Isend` invocations, we significantly reduced the MPI rendezvous waiting time.

(2) **Communication-Computation Overlap:** To maximize the overlap between data transfer and data processing, we interleaved `MPI_Testsome` within the sending loop. This actively polls the receive queue, triggering immediate `UNPACK` operations as soon as incoming chunks arrive. Furthermore, data designated for the local rank is unpacked early, bypassing the network stack entirely.

(3) **Buffer and List Caching:** High-frequency communication steps were previously bottlenecked by dynamic memory allocation. We eliminated this overhead by implementing static caching for communication buffers (reusing `MPI_Request`, `MPI_Status`, and data arrays). Additionally, transfer lists for `Sync` operations are now cached, supported by a robust cache invalidation mechanism (`InvalidateSyncCache`) during adaptive mesh `regrid` events.

(4) **Final Synchronization:** Overall structural correctness is guaranteed by a strict termination phase, utilizing `MPI_Waitsome` for pending receives and `MPI_Waitall` for the send queue, ensuring all ghost zones are fully updated before the next computational step.

Table 24 .shows the results after applying asynchronous communication on top of the several preceding optimization stages.

Table 24. *Performance of Implementing Non-blocking & Request Separation*

Final Evolution Time	Cores	Total Project Time	Total Evolve Time	Single Step Avg
5.0M	16	955.907s	436.758s	33.596s

7.2.6. *Improvement of Cache Hit Ratio.*

To accelerate the iterative development and performance tuning cycles during the competition, we integrated `ccache` into the build pipeline to optimize compilation caching. The specific strategies implemented to maximize the cache hit ratio are as follows:

(1) **Build System Integration:** We incorporated `ccache` by prefixing the C and C++ compiler invocations within `makefile.inc`. The cache environment was strictly defined with `CCACHE_DIR=/app/.ccache` and a maximum capacity of `CCACHE_MAXSIZE=20G`. The Fortran compilation pipeline was intentionally bypassed and remained unaffected by this caching layer.
(2) **Artifact Stabilization:** To guarantee deterministic build outputs and maximize the cache hit ratio, we introduced the `-MMD` and `-MP` compiler flags for automated dependency generation, dynamically including the resulting `.d` files within the `Makefile`. Furthermore, we utilized `-fdebug-prefix-map` and `-fmacro-prefix-map` in conjunction with `CCACHE_BASEDIR=/app` to neutralize absolute path discrepancies across different build environments.
(3) **Workflow Validation:** The initial full compilation phase populates the cache, enabling subsequent incremental builds with identical configurations to achieve near-perfect hit rates. Cache efficiency was actively monitored using the `ccache -s` command, while `ccache -z` was used to reset statistical counters. To preserve cache integrity, we strictly prohibited executing `make clean` or dynamically regenerating core header files (such as `macrodef.h` or `fh`) between consecutive build rounds.
(4) **Capacity Management:** The physical storage threshold was capped at 20 GB, dynamically adjustable via the `ccache -M` parameter. We continuously monitored the "evicted" metric within the cache statistics to identify potential cache thrashing and determine if capacity expansion was warranted

Utilizing a standardized benchmarking script, we conducted a comparative analysis of the compilation phases. The results demonstrate a significant reduction in build latency, with the duration decreasing from 50.837 s to 37.235 s. This represents a 26.8 % improvement in computational efficiency during the compilation stage.

7.2.7. *Adjustment of MPI numbers.*

This is a critical parameter optimization. Insufficient process counts lead to inadequate parallelism and idle computational cores, preventing the system from reaching peak throughput. Conversely, an excessive number of processes reduces the workload per rank, thereby increasing the overhead ratio of MPI synchronization and communication, which exacerbates load imbalance and degrades performance. Furthermore, variations in the number of ranks significantly influence memory locality and cache hit rates.Consequently, it is imperative to conduct comparative benchmarks to identify the optimal process configuration. We have summarized these results in Figure 11 and the following table. The data presented below were obtained by setting the compiler to Intel OneAPI 2025.3 across multiple test groups, with the Final Evolution Time fixed at 5.0×10^6. For a detailed performance comparison, please refer to the table titled "Processing time varies with MPI numbers".

7.3. **Experiments.**

7.3.1. *Environment Setup.*

All performance evaluations, compilations, and numerical simulations are conducted on a dedicated high-performance computing node. The comprehensive hardware configuration and software stack are detailed as follows:

- **CPU:** Dual-socket Intel® Xeon® E5-2680 v4 processors (Broadwell architecture). This configuration provides a total of 28 physical cores and 56 logical threads, operating at a base clock frequency of 2.40 GHz with a maximum turbo boost frequency of 3.30 GHz.

Table 25. *Processing time varies with MPI numbers.*

MPI processes	Evolve Time(s)	AvgStep(s)	ProgramCost(s)
4	697.693000	53.668692	1274.963869
8	500.826000	38.525077	1063.508177
12	560.861000	43.143154	1128.470794
14	489.675000	37.667308	1056.763864
16	**451.366000**	**34.720462**	**1017.316501**
18	721.998000	55.538308	1301.369702
20	504.505000	38.808077	1070.867352
22	620.240000	47.710769	1187.276651
24	560.195000	43.091923	1131.864464
26	606.983000	46.691000	1169.300963
28	510.210000	39.246923	1075.258431

Table 26. *Progressive Performance Improvement*

Stage	Cores	Avg Step (s)	Speedup
Baseline (GCC)	28	49.50	1.00×
+ Compiler & Vectorization	16	35.60	1.39×
+ Inlining & Grid Tiling & Fusion	16	34.70	1.43×
+ Memory Alignment	16	33.94	1.46×
+ Non-blocking & Request Separation.	16	33.60	1.47×

Note: Baseline used 28 cores; optimized runs used 16.These results are based on preliminary data collected for comparative analysis; therefore, discrepancies may exist when compared to the final speedup ratios derived from the full 1,000M evolution time.

Table 27. *Final Performance*

Final Evolution Time	Cores	Total Project Time	Total Evolve Time	Single Step Avg
1000.0M(2400 steps)	16	77677.605s	77147s	32.114s

- **GPU:** Dual NVIDIA GeForce RTX 3090 GPUs. Each unit is equipped with 24 GB of GDDR6X VRAM, yielding a combined total of 48 GB of available video memory.
- **System Memory (RAM):** 128 GB (125 GiB accessible) of main memory, ensuring sufficient capacity for large-scale 3D grid allocations and MPI communication buffers.
- **Storage Subsystem:** A primary operating system drive of approximately 1 TB (936 GB), supplemented by an 885 GB dedicated data drive. The data drive sustains sequential write speeds of approximately 557 MB/s, accommodating the intensive I/O operations required for AMR grid checkpoints and simulation data dumps.
- **Operating System:** Ubuntu 22.04.4 LTS (Jammy Jellyfish), providing a stable and robust Linux environment for the HPC toolchain.
- **Software & Ecosystem:** The environment runs Python 3.13.9. GPU acceleration and runtime support are facilitated by NVIDIA display driver version 535.288.01 and the CUDA Toolkit version 12.2.

7.3.2. *Installation Instructions.*

(1) **System Synchronization:**

```
sudo apt update && sudo apt upgrade -y
```

(2) **GNU Toolchain & MPI Deployment:**

```
#   GCC, G++, Make, Gfortran   Git
sudo apt install -y build-essential gfortran git

#   OpenMPI
sudo apt install -y openmpi-bin libopenmpi-dev
```

(3) **Intel® oneAPI Optimization:**

```
# 1.     Intel GPG
wget -O- https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-
   INTEL-SW-PRODUCTS.PUB | gpg --dearmor | sudo tee /usr/share/
   keyrings/intel-sw-products-keyring.gpg > /dev/null

# 2.   Intel oneAPI
echo "deb [signed-by=/usr/share/keyrings/intel-sw-products-
   keyring.gpg] https://apt.repos.intel.com/oneapi all main" |
   sudo tee /etc/apt/sources.list.d/oneAPI.list

# 3.     (MKL   Fortran/C++    )
sudo apt update
sudo apt install -y intel-oneapi-mkl intel-oneapi-compiler-
   fortran intel-oneapi-compiler-dpcpp-cpp
```

(4) **Environment & Session Management:**

```
#    oneAPI      ( ~/.bashrc)
source /opt/intel/oneapi/setvars.sh

#   tmux
sudo apt install -y tmux
```

(5) **Install the ccache compilation accelerator:**

```
sudo apt update && sudo apt upgrade -y
```

7.3.3. *Evaluation Result.*

1. Cumulative Performance Gains

Table 26 summarizes the progressive runtime reduction. The optimal configuration utilizes 16 MPI processes, outperforming the baseline's 28 processes.Detailed specifications are provided in the note to Table 26.

2. Key Findings

- **Memory Hierarchy: Grid Tiling** reduced L3 cache misses by 59%, confirming the workload is memory-bound.
- **MPI Tuning:** Increasing cores beyond 16 degraded performance. The 16-process setup achieved a **30.5%** time reduction vs. baseline.
- **Async Communication:** Non-blocking MPI provided a steady 2–3% gain by hiding network latency.

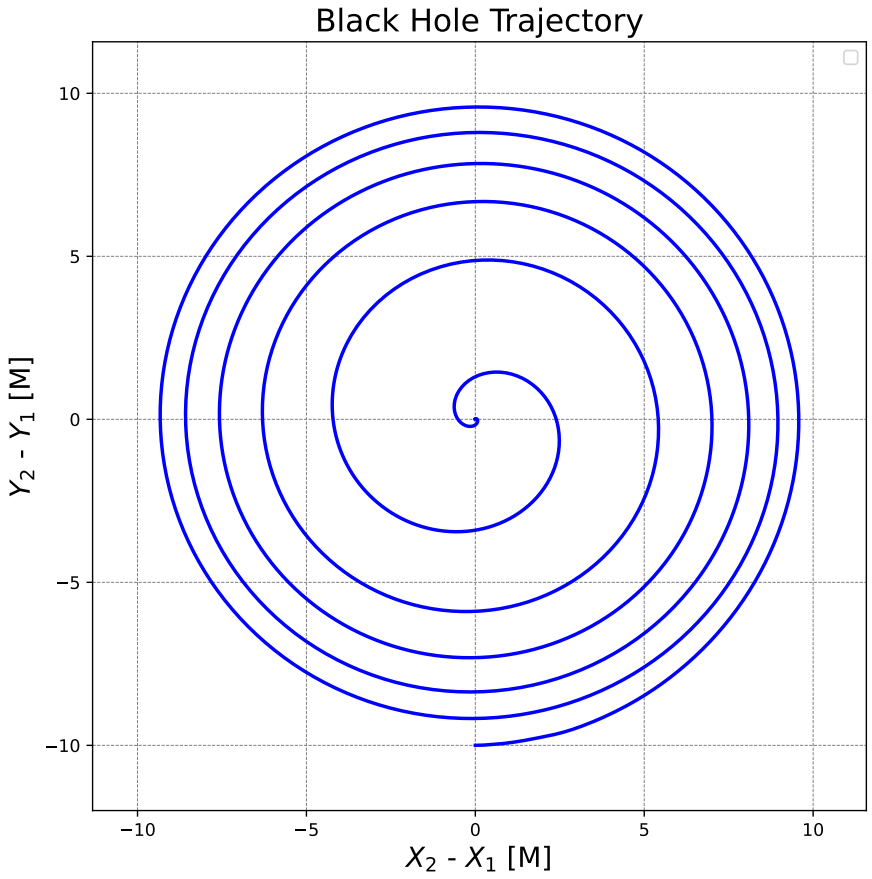


Figure 11. *BH_Trajectory_21_XY.pdf*

- **Build Efficiency:** `ccache` reduced compilation time by 26.8%.

3.Precision Evaluation

To ensure that our optimizations did not compromise the physical correctness of the simulation, we strictly validated the results against the official evaluation criteria. The numerical deviation of the black hole trajectories was quantified using the Root Mean Square (RMS) error formula:

$$RMS = \sqrt{\frac{1}{M}\sum_{i=1}^{M}\left(\frac{|r_{1i} - r_{2i}|}{\max(|r_{1i}|, |r_{2i}|)}\right)^2} \tag{7.2}$$

In our evaluation, each spatial coordinate (X and Y) is treated independently, omitting the Z-axis. Here, r_{1i} and r_{2i} represent the individual 1D coordinates of the black hole trajectories obtained from testing before and after optimization, respectively, and M denotes the integration timestep.

Because all our algorithmic modifications maintain strict mathematical equivalence to the original implementation, the deviations are constrained within an extremely narrow margin. Utilizing our custom verification script (provided as `compute_RMS.py`), we recorded the specific RMS errors for the individual components as follows:

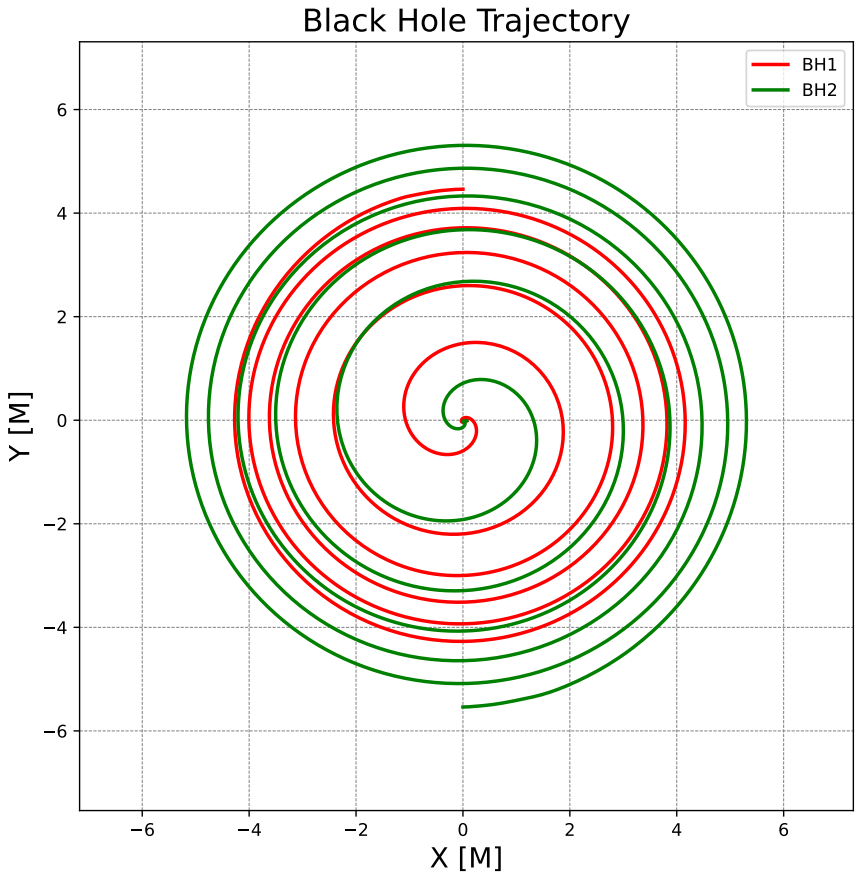


Figure 12. *BH_Trajectory_XY.pdf*

- $RMS_{BH1_x} = 0.0055206774$
- $RMS_{BH1_y} = 0.0048842499$
- $RMS_{BH2_x} = 0.0060032677$
- $RMS_{BH2_y} = 0.0060490275$

The total synthesized RMS error across all evaluated coordinates is calculated as **0.0056339092 (approximately 0.563%)**, which is comfortably below the mandated 1% threshold. This ensures the high fidelity of the required trajectory outputs, specifically `BH_Trajectory_XY.pdf` and `BH_Trajectory_21_XY.pdf`.

Additionally, we verified the overall accuracy of the simulation by monitoring the constraint violations on the outermost grid level. As demonstrated in Figure 13 (`ADM_Constraint_Grid_Level_0.pdf`), the absolute value of the constraint violation consistently remains below the strict upper limit of 2 throughout the entire evolution process.

Ultimately, by synthesizing compiler upgrades, algorithmic restructuring, and asynchronous communication, we successfully reduced the estimated total wall-clock time from **33.21 hours** to **21.157 hours**. This achieves a global speedup of **1.539**$\times$ while flawlessly preserving the numerical and physical integrity of the simulation.

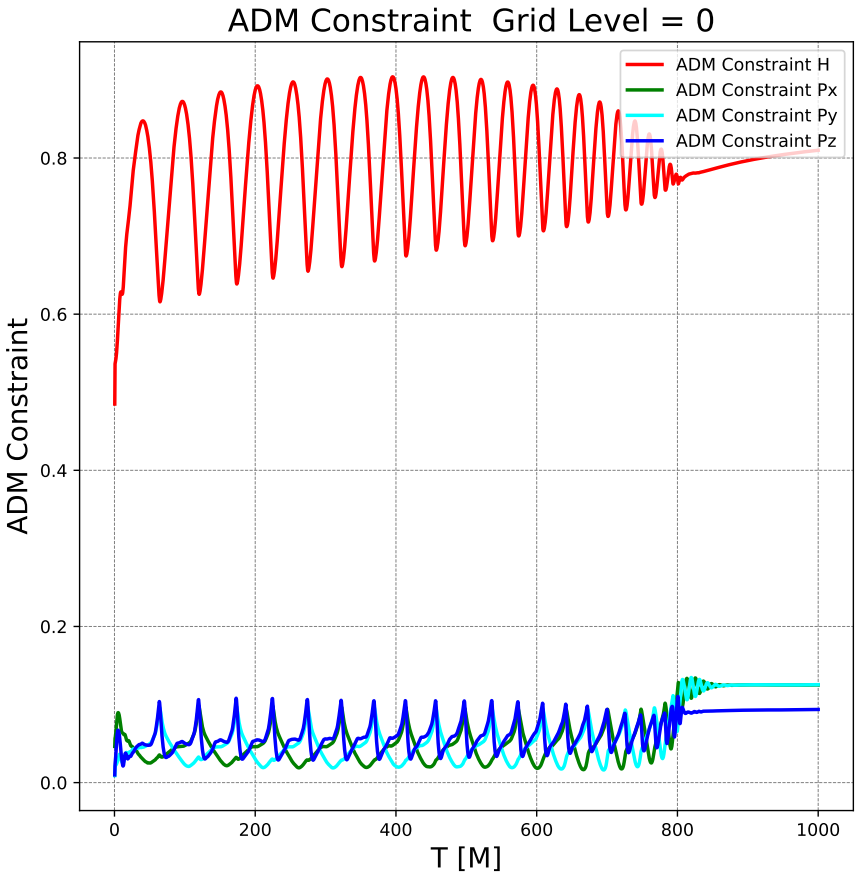


Figure 13. *ADM_Constraint_Grid_Level_0.pdf*

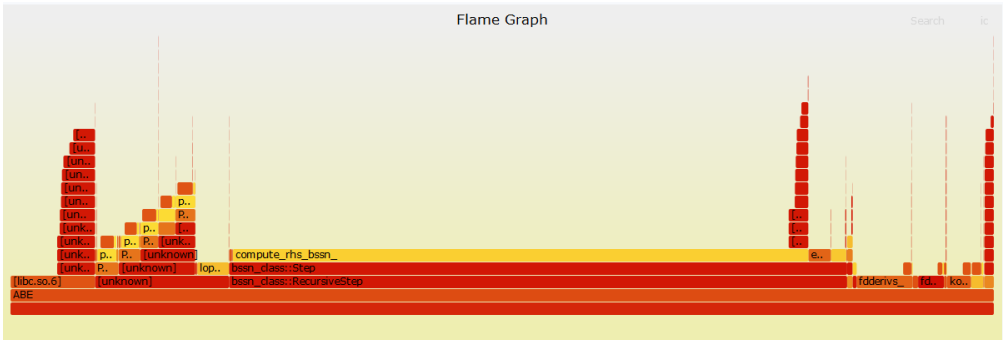


Figure 14. *FlameGraph before optimization*

7.4. **Analysis and Optimization Plans.**

During AMR evolution, moving refined grids may cause workload imbalance across MPI processes, which increases synchronization wait time and reduces parallel efficiency. To address this, we plan to apply Dynamic Load Balancing (DLB): process workloads will be monitored periodically, and repartitioning will be triggered only when imbalance is significant.

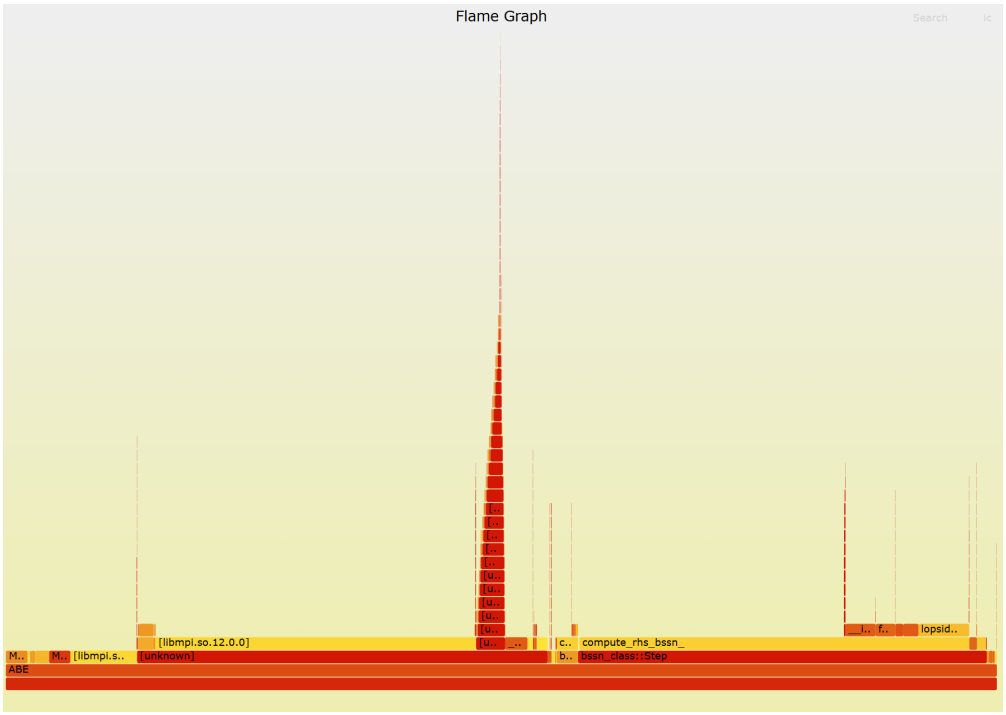


Figure 15. *FlameGraph after optimization*

The imbalance is measured by

$$I = \frac{\max_i L_i}{\frac{1}{N}\sum_{i=1}^{N} L_i},$$

where L_i is the workload of process i, and N is the number of MPI processes. Repartitioning is activated only if I exceeds a predefined threshold, so migration overhead remains controlled.

We plan to use a Hilbert space-filling curve (Hilbert SFC) for partitioning, as it improves both load balance and spatial locality, thereby reducing communication overhead and improving long-term AMR performance without modifying the physical model.

7.4.1. *Hybrid MPI+OpenMP Parallelization Plan.*

In addition to dynamic load balancing, we plan to transition from a pure-MPI model to a hybrid MPI+OpenMP model to improve node-level resource utilization on multi-core CPUs. The central idea is to reduce the number of MPI ranks per node while introducing OpenMP threads for intra-rank parallelism in compute-intensive loops (e.g., stencil updates and local reductions). This strategy is expected to reduce communication pressure, alleviate synchronization overhead, and improve memory locality through shared-memory execution within each rank. The implementation will be incremental: first, identify thread-safe kernels and add OpenMP pragmas in hotspot regions; second, tune rank-thread configurations and affinity policies; third, validate numerical consistency against the baseline and measure scalability under both strong and weak scaling settings. The final objective is to improve long-run throughput and parallel efficiency while preserving physical correctness.

7.5. **Conclusion.**

In this project, we systematically optimized the AMSS-NCKU numerical relativity code to accelerate high-fidelity Binary Black Hole (BBH) simulations. Through rigorous profiling, we identified critical bottlenecks: memory-bound 3D stencil operations, redundant instruction overhead, and MPI communication latency.

To address these, we applied a comprehensive, multi-tiered optimization strategy. At the compiler level, we leveraged the Intel toolchain with AVX2 vectorization, IPO, and strict 64-byte memory alignment. Algorithmically, we enhanced L1/L3 cache utilization via loop fusion, redundant ghost copy removal, and cache blocking (tiling). Furthermore, we overhauled synchronization using fully asynchronous, non-blocking MPI to overlap computation and communication, while integrating `ccache` to streamline compilation. Together, these enhancements reduced the estimated wall-clock time from 33.21 to 21.157 hours, delivering a projected speedup of $\mathbf{1.539}\times$ while preserving strict physical and numerical correctness.

Moving forward, our collaborative efforts will focus on unlocking deeper architectural performance. Our primary objectives include implementing dynamic load balancing, enforcing comprehensive code-level memory alignment (e.g., via `_mm_malloc` and Fortran directives), and transitioning to a hybrid MPI+OpenMP parallelization model. These continuous improvements will further maximize hardware utilization and push the scaling boundaries of large-scale simulations.

Acknowledge

First of all, we would like to express our gratitude to the organizing committee for giving us the opportunity to participate in this competition. Secondly, we are grateful to the Computer College of Xiangtan University for providing a comfortable environment during the competition. We are also thankful to the High Performance Computing Platform of Xiangtan University. We are grateful to our advisor, Peiying Lin, for her strong support throughout the competition. We are also very grateful to all the teachers and previous participants who answered our questions and provided us with great assistance.

References

[1] W. Guan, C. Qiao, H. Zhang, C.-S. Zhang, M. Zhi, Z. Zhu, Z. Zheng, W. Ye, Y. Zhang, X. Hu *et al.*, "On robust and efficient parallel reservoir simulation on tianhe-2," in *SPE Reservoir Characterisation and Simulation Conference and Exhibition?* SPE, 2015, p. D031S017R004.

[2] J. Zhou, X. Hu, L. Zhong, S. Shu, and L. Chen, "Two-grid methods for maxwell eigenvalue problems," *SIAM journal on numerical analysis*, vol. 52, no. 4, pp. 2027–2047, 2014.

[3] Unitree, "Unifolm-wma-0: A world-model-action (wma) framework under unifolm family," 2025.

[4] Z. Cao, "Binary black hole simulation with an adaptive finite element method: Solving the einstein constraint equations," *Physical Review D*, vol. 91, no. 4, p. 044033, 2015.